

JC2001 软件工程导论

小组项目 · 技术报告

学年 2026-27

Campus Market: 面向高校校园的二手交易平台设计与实现

面向校内学生买卖与交换闲置物品的概念验证型 Web 应用

组号: 9

专业: 人工智能专业 理学学士 (BSc (Artificial Intelligence))

组员	姓名	学号
组员 1	<姓名>	<学号>
组员 2	<姓名>	<学号>
组员 3	<姓名>	<学号>
组员 4	<姓名>	<学号>
组员 5	<姓名>	<学号>

提交至 MyAberdeen 课程作业区 · 提交日期: <date>

目录

目录.....	2
图目录.....	6
表目录.....	7
1 引言与项目背景.....	9
1.1 背景.....	9
1.2 问题.....	9
1.3 项目目标.....	10
1.4 目标用户与利益相关方.....	10
1.5 概念验证的范围.....	10
1.6 主要功能.....	11
1.7 实际价值与业务价值.....	11
1.8 报告结构.....	12
2 项目方法与过程.....	13
2.1 开发过程.....	13
2.2 需求获取.....	13
2.3 工具与配置管理.....	14
2.4 团队组织.....	14
2.5 约束与假设.....	14
2.6 如何验证工作.....	15
3 需求与用例.....	16
3.1 需求建模方法.....	16
3.2 功能需求.....	16
3.3 非功能需求.....	17
3.4 需求优先级.....	18
3.5 系统上下文.....	18
3.6 用例模型.....	19

3.7 用例详细描述.....	21
3.8 业务流程模型.....	24
3.9 发布活动的活动模型.....	25
3.10 需求追溯.....	25
4 系统设计与架构.....	26
4.1 架构风格与被考虑的替代方案.....	26
4.2 分层架构.....	26
4.3 模块划分.....	27
4.4 领域模型.....	28
4.5 数据库设计.....	29
4.6 交易状态机.....	31
4.7 交互设计.....	32
4.8 接口设计.....	34
4.9 客户端设计.....	36
4.10 设计决策与取舍.....	37
5 实现.....	39
5.1 技术栈.....	39
5.2 软件结构与规模.....	39
5.3 账号、会话与凭据存储.....	40
5.4 商品目录查询.....	40
5.5 照片上传与存储.....	41
5.6 状态机.....	41
5.7 会话列表查询.....	42
5.8 客户端实现.....	43
5.9 设计体系.....	43
5.10 用户界面.....	44
5.11 编码规范与约定.....	50

5.12 部署与可复现性.....	51
6 测试与评估.....	52
6.1 测试策略.....	52
6.2 测试环境与测试框架.....	52
6.3 单元与组件测试结果.....	53
6.4 接口测试.....	54
6.5 反向与边界测试.....	54
6.6 状态机测试.....	54
6.7 权限测试.....	55
6.8 需求覆盖.....	55
6.9 性能测试.....	56
6.10 扩展性测量.....	57
6.11 界面与可用性评估.....	58
6.12 非功能需求的验证.....	58
6.13 发现并修复的缺陷.....	58
6.14 效度威胁与未测试的内容.....	59
7 结论与后续工作.....	60
7.1 对照目标的完成情况.....	60
7.2 局限.....	60
7.3 后续工作.....	61
7.4 对过程的反思.....	61
7.5 结语.....	62
参考文献.....	63
附录 A 小组工作量表.....	65
A.1 定量贡献.....	65
A.2 定性贡献.....	65
A.3 声明.....	65

附录 B 需求追溯矩阵.....	67
附录 C 验证明细.....	69
C.1 接口契约（表 6.3）	69
C.2 状态机迁移覆盖（表 6.5）	69
C.3 权限矩阵（表 6.6）	70
C.4 非功能需求的验证（表 6.9）	70
C.5 缺陷日志（表 6.10）	71

图目录

图 3.1	系统上下文：概念验证所交互的参与者、浏览器与本地文件系统	19
图 3.2	用例图：两个参与者下的 18 个用例，访客参与者泛化为注册学生	20
图 3.3	一笔完整二手交易的业务流程模型，服务端守卫显示在系统泳道中	24
图 3.4	发布商品的活动模型，含三处判断与前后端重复执行	25
图 4.1	分层架构：应用层的接口分组与数据层的组件	27
图 4.2	模块划分与依赖方向	28
图 4.3	领域类图：六个实体及其属性与关联	29
图 4.4	六张表的实体关系图，含外键与基数	30
图 4.5	商品状态机：每条迁移的参与者与守卫条件	31
图 4.6	发布商品的时序图：从提交表单到写入数据行	32
图 4.7	预订商品与确认成交的时序图，含两处守卫与拒绝路径	33
图 4.8	会话列表、打开会话与回复的时序图	34
图 4.9	按资源分组的 JSON 接口一览，含方法与会话要求	35
图 4.10	前端视图路由：五个视图、常驻页头，以及需要会话的视图上的守卫	37
图 5.1	技术选型与理由	39
图 5.2	状态变更请求在写库前的校验流程，由一张五条目的规则表驱动	42
图 5.3	用一条聚合查询而非按会话各查一次来渲染会话列表	42
图 5.4	商品目录网格：每张卡片上的状态徽章、封面图、价格、卖家与分类	44
图 5.5	搜索无结果的空状态：说明情形并给出下一步，而不是留下空白网格	45
图 5.6	已登录买家看到的详情页：预订、收藏与联系卖家，以及卖家与结构化属性	46
图 5.7	同一界面对卖家：购买操作被管理操作取代	46
图 5.8	界面中所有破坏性操作共用的确认弹窗	47
图 5.9	发布表单：分类与成色的芯片选择器、照片拖放区，以及挂在出错字段上的校验提示	48
图 5.10	个人中心：带统计数的资料摘要，以及学生本人的发布	48
图 5.11	打开的会话：区分本人与对方的消息、相对时间，以及固定在底部的输入框	49
图 5.12	400 像素视口下的界面：网格收敛为两列，页头缩减为必要控件	50
图 6.1	按组件划分的自动化测试用例。80 个用例全部通过。	53
图 6.2	按类别统计已实现并已独立验证的需求数	55
图 6.3	500 件商品规模下各操作的进程内响应时间，中位数与 95 分位	56
图 6.4	商品数量从 100 增长到 2000 时三条目录读取路径的中位响应时间	57

表目录

表 1.1	利益相关方及其对系统的诉求	10
表 1.2	概念验证的范围边界	10
表 1.3	概念验证的主要功能及其需求与验证位置	11
表 2.1	角色与职责。个人贡献与商定的百分比权重记录在附录 A 的小组工作表中。	14
表 2.2	项目约束与假设	14
表 3.1	功能需求	16
表 3.2	非功能需求及其验证方式	17
表 3.3	需求优先级	18
表 3.4	用例汇总	20
表 4.1	架构替代方案及各自的否决理由	26
表 4.2	各层职责，以及各层被禁止做的事	27
表 4.3	模块清单	28
表 4.4	数据表规格	30
表 4.5	状态迁移规则。角色与原状态两者都必须满足；任一不满足的请求不改变任何数据。	31
表 4.6	接口清单。标记「需要」表示未登录时直接返回 401，且不解析请求体。	35
表 4.7	所有接口共用的响应信封	36
表 4.8	设计决策日志	37
表 5.1	组件版本与作用	39
表 5.2	所交付软件各部分的规模	40
表 5.3	上传与发布所应用的校验规则及各自的执行位置	41
表 5.4	客户端各渲染路径及其所应用的转义	43
表 5.5	设计令牌分组及其控制的内容	43
表 5.6	在整个代码库中应用的编码约定	50
表 6.1	测试层次、技术与检出能力	52
表 6.2	按分组划分的自动化测试套件	53
表 6.4	刻意编写的反向与边界用例	—
表 6.7	各操作的进程内响应时间。数据集：500 件商品、22 个账号、61 个会话、一个 40 条消息的会话。Python 3.13.14，读操作重复 200 次，写操作重复 100 次。	56
表 6.8	数据集增长时三条目录读取路径的中位响应时间	57
表 7.1	各项目标的完成情况	60
表 7.2	所交付概念验证的局限	60
表 7.3	按优先级排列的后续工作	61

1 引言与项目背景

1.1 背景

每个学年都有大量仍可使用的物品在学生之间流转。只用一个学期的教材、升级后换下来的笔记本和显示器、自行车、计算器、运动装备以及厨房用品，在原主人不再需要之后往往闲置在宿舍里；而对这些物品的需求，就在几百米外的下一个年级。关于二手交易的消费研究表明，阻碍交易的主要因素不是价格，而是摩擦：发布信息的成本、不确定是否有人需要，以及约见买家本身的麻烦。在大学校园里，物理距离可以忽略，剩下的摩擦几乎全是信息层面的——这正是本项目要解决的问题。

项目所要规范化的行为其实已经存在：学生确实在把教材和电子设备传给彼此。真正值得追问的是，为什么他们要通过并不适合这件事的渠道来做，以及一个专用应用能否做得明显更好。

1.2 问题

目前承载校园交易的主要有三类渠道，而它们各自以不同方式失效。

- **通用二手平台。** 它们面向的是互不相识、地理上分散的买卖双方。校内商品会被同城结果淹没；需要和一个可能在一小时车程之外的人协商见面；平台还要抽佣或售卖推广位。对一位只想卖掉 40 元教材的学生来说，净收益接近于零，因此这个渠道根本不会被使用。
- **社交媒体群组与群聊。** 绝大多数交易实际发生在这里，但聊天流是一个糟糕的商品目录：一条信息几小时内就被刷走，没有搜索、没有筛选、无法判断商品是否还在，也没有任何成交记录。因为没有唯一的商品条目，同一件教材会被不同人反复重发。
- **公告栏与口口相传。** 只能触达碰巧路过的人，买家也无法在跑一趟之前比较价格或判断成色。

其中有两个失效是校园场景特有的，也解释了为什么不能简单地把通用平台指向校园。

第一是**信任**：学生交易的对象可能是同一间教室里的人，但彼此并不认识；而一个对全网开放的平台无法给出「对方是同一所学校的成员」这一信号，也没有约定的公共见面地点。第二——也是本项目视为核心设计问题的一点——是**状态**。在非正式渠道中，「这件东西还在吗」没有权威答案：卖家要重复回答几十次，买家会为昨天已经卖掉的东西白跑一趟，双方都无法判断某件商品是仍在售、已被预留，还是已经售出。

由此造成的后果是双向的低效：卖家把卖不掉的物品丢掉或送人，一分钱都收不回来；买家因为从未看到二手选择，只能按原价买新的；而学校则要处理一批本可以被自己的学生买走的可用物品。解决这个问题之所以值得，一是经济上直接惠及价格敏感度最高的学生群体，二是环境上每一次复用都意味着少制造一件、少丢弃一件，三是学校层面能减少学年末被丢弃的可用物品数量。

1.3 项目目标

本项目设定了五个目标，每一个都以「可被证明完成」而非「可被声称完成」的方式表述。

- (1) **明确校园二手交易平台的需求：**识别其用户与利益相关方，产出一组有优先级的功能与非功能需求，作为后续所有工作的追溯基准。
- (2) **设计系统架构与数据模型：**以分层架构、领域模型、关系模式和一组交互模型来表达设计，并记录关键设计决策及其被否定的替代方案。
- (3) **实现可运行的概念验证，**端到端演示核心功能。
- (4) **依据所声明的需求验证概念验证：**通过自动化单元、组件与接口测试，针对性的反向与边界测试，性能测量，以及界面评估来完成验证，并如实报告结果，包括失败或仍未被验证的部分。
- (5) **交付课程要求的文档与演示材料，**并保证其与所交付的软件一致。

1.4 目标用户与利益相关方

表 1.1 利益相关方及其对系统的诉求

利益相关方	与系统的关系	核心诉求
访客（未注册学生）	直接用户。通过链接或搜索进入站点，尚无账号。	浏览与搜索商品、完整查看商品页，并在决定是否注册前对卖家有足够了解。
注册学生	直接用户，同时扮演两种角色：同一个账号既卖也买，角色仅在交易过程中区分。	作为卖家：快速发布、可被联系、商品售出后不再收到询问。作为买家：找到在售商品、确知是否仍在售、并能联系卖家。
学校	间接相关方。提供校园、学生，在生产环境中还提供托管。	平台不涉及资金、只保存最少的个人信息、不产生运维成本或合规义务。
课程评阅人	间接相关方。安装、运行并评估本软件。	软件能在干净机器上无需专业知识即可安装运行，且实现与报告所述一致。

1.5 概念验证的范围

表 1.2 概念验证的范围边界

范围内	刻意排除在范围外
-----	----------

范围内	刻意排除在范围外
账号注册、身份认证与会话管理	校内统一身份认证，以及与学生数据库的实名核验
商品发布，最多三张照片与一组结构化属性	富媒体（视频）、批量导入，以及从其他平台的目录同步
商品目录浏览，含关键词搜索、分类与状态筛选、三种排序与分页	推荐、个性化与相关性排序
商品详情页，含图廊、描述、卖家信息与公开留言	信誉评分、评价体系与纠纷记录
收藏、私信与个人中心	邮件或推送通知，以及实时投递
服务端校验的交易状态：预订、确认成交、取消、下架、重新上架	在线支付、担保交易、物流与退款
单机部署	水平扩展、高可用与多租户运营

1.6 主要功能

下表列出已实现并演示的功能。每一项都对应第 3 章的功能需求分组，以及第 6 章中验证它的测试用例。

表 1.3 概念验证的主要功能及其需求与验证位置

功能	作用	对应需求	验证方式
账号与会话	注册、登录、登出、获取本人资料；密码以加盐哈希存储	FR-01 至 FR-04	TC-A01 至 TC-A10
商品发布	创建、编辑与软删除商品，最多三张照片，所有必填字段均做校验	FR-05 至 FR-10	TC-L01 至 TC-L12、TC-U01 至 TC-U08
商品目录	分页商品网格，两列关键词搜索、分类与状态筛选、三种排序	FR-11 至 FR-15	TC-C01 至 TC-C12
商品详情	图廊、描述、卖家信息、结构化属性、留言区与按角色区分的操作	FR-16 至 FR-18	TC-C13、TC-C14、TC-S05 至 TC-S08
收藏	收藏与取消收藏；个人中心中的收藏列表	FR-19 至 FR-21	TC-S01 至 TC-S04
交易状态	预订、确认成交、取消预订、下架与重新上架，每一步都由角色与当前状态双重把关	FR-22 至 FR-24	TC-T01 至 TC-T15
私信	发送私信、按对象汇总会话列表、打开会话并清除未读	FR-25 至 FR-27	TC-S09 至 TC-S16
个人中心	我的发布、我的收藏、已售出与我的会话集中在一处	FR-28	TC-P01 至 TC-P05

1.7 实际价值与业务价值

本系统的价值来自消除信息摩擦，而不是提供某种全新能力。学生本来就想做二手交易，他们缺的是一个让信息留得住、搜得到，并且能权威地回答「是否还在售」的地方。

对学生而言，价值可以直接折算成钱和时间：一本教材半价转手，买家省下一半书费，卖家收回一半成本；在一个几百人的年级里，累积效果相当可观。两端的时间也都省了下来——买家去搜索，而不是沿着公告栏走一圈；卖家只需公开回答一次「还在吗」，而不是每来一个人回答一次。对学校而言，校园二手市场减少了学年末被处理的可用物品，并提供了一项零运营成本的学生服务：因为软件本身不涉及资金，也就不需要抽佣、不受金融监管、不保存任何财务数据。

同时也要说清楚价值**不在哪里**。本平台并不从交易中获利，也不存在可行的变现路径——对一本 40 元的教材抽佣，会直接把使用动机消灭掉。它的价值在于社区层面以及学校在减废与学生体验方面的目标，这也正是设计上优先考虑「易被采用」和「低运营成本」而不是「收入」的原因。

1.8 报告结构

第 2 章说明所用过程及其约束条件。第 3 章给出功能与非功能需求、用例模型以及业务流程与活动模型。第 4 章呈现架构、领域与数据模型、交互模型和设计决策，每一项都附上被否定的替代方案。第 5 章描述实现，包括技术选型、软件结构、工程风险最高的算法以及用户界面。第 6 章报告测试与评估：80 个自动化用例的结果、逐操作的性能测量、数据规模扩展测量，并明确说明测试未覆盖的部分。第 7 章对照第 1.3 节的目标作结论，并列出局限与后续工作。附录 A 是小组工作量表，附录 B 是完整的需求追溯矩阵，附录 C 是验证明细。

2 项目方法与过程

2.1 开发过程

项目采用**增量式、以原型驱动**的过程，而不是一次性瀑布流程，也不是完整的敏捷仪式。理由很具体：团队有固定的截止日期和固定的成员，而范围在开始时并不清楚——真正最难确定的需求（两个买家同时看中一件商品时会发生什么）从需求访谈里根本看不出来。先做一个窄而完整的端到端原型，把这条需求变成具体的东西，第 3 章的需求规格是在拿到原型之后才定稿的。

- **按纵向切片而不是按层推进。** 每个增量都贯穿界面、接口、查询与存储各层，交付一项用户可见的能力，而不是逐层完成。这使软件在任何时刻都可运行，也避免了在最后阶段一次性集成多个未验证的层。
- **需求规格在首个原型之后稳定下来。** 早期增量帮助厘清需求，之后的增量则严格按需求编号开发，因此每项功能都能向上追溯到需求、向下追溯到测试。
- **测试与代码同步编写。** 第 6 章描述的测试套件随实现一起成长，每次改动后都会运行；这正是后期一次全站前端重做没有引入回归的原因。
- **单一仓库、频繁提交。** 所有工作都通过 Git 进行，因此每次改动都可归属、每个增量都可恢复。

2.2 需求获取

需求来自五种来源。它们提供的信息类型各不相同而非刻意求全；其中最有价值的反而是最不正式的那种——观察一次真实的二手交易究竟在哪里卡住。

- **半结构化访谈：**覆盖两个年级的 6 名学生，选取标准是上一年买过或卖过二手物品。由此得到现有渠道的失效方式，以及「这件还在吗」是主要交互成本这一结论。
- **文档与实物分析：**对 4 个活跃的校园转卖群采样两周，得到重复发布问题的证据，以及界面中作为固定选项提供的分类词表。
- **任务观察：**跟踪 3 件商品从发布到交付的全过程，得到真实交易的步骤序列，即第 3.8 节的业务流程模型。
- **与指导教师讨论：**两次约定会议，产出范围削减决定——砍掉支付、配送与信誉体系，这三项中任何一项都会让项目无法交付。

- **竞品观察**：浏览两个通用二手平台的公开商品，得到买家筛选商品时真正依赖的属性：价格、成色与照片。

2.3 工具与配置管理

- **Git 与 GitHub** 用于版本控制，使每次改动可归属、每个增量可恢复。
- **Python 3.8 及以上** 作为实现语言，因为机房与教师机上已经预装，且无需采购运行时授权。
- **Flask 2.3.3** 作为 Web 框架，路由、签名会话与 JSON 处理都由一个小依赖提供，无需额外配置。
- **SQLite 3** 作为数据库，因为它是文件型的，数据库模式随作业一起提交。
- **pytest** 用于自动化测试，其 fixture 机制天然支持每个用例一套独立数据库，无需引入额外库。
- **Playwright 驱动 Microsoft Edge** 用于界面验证，复用机器上已有的浏览器，不必再下载一个。
- **Microsoft Word** 用于文档制作，这也是课程要求的输出格式。

2.4 团队组织

工作按**功能纵向**划分，而不是按技术层次划分。若按层分工（一人做数据库、一人做接口、一人做界面），结果是三个人互相等，直到项目后期才有东西能跑；按功能划分则让每位成员都拿到一个可以独立完成并演示的切片。此外由一位轮换的评审人检查每次改动是否满足它所声称对应的需求。

表 2.1 角色与职责。个人贡献与商定的百分比权重记录在附录 A 的小组工作量表中。

角色	成员	职责
需求负责人	组员 1	需求获取、功能需求、需求优先级与追溯矩阵
架构负责人	组员 2	架构选型、模块边界、接口契约、与指导教师沟通
数据与安全负责人	组员 3	模式设计与范式化、密码存储、上传校验、性能测量
流程与状态负责人	组员 4	交易状态机、权限规则、反向与边界测试设计
界面负责人	组员 5	设计体系、全部视图与反馈状态、响应式行为、用户手册
评审人（轮换）	全体成员	改动合并前，核对其是否满足所声称对应的需求

2.5 约束与假设

表 2.2 项目约束与假设

编号	约束或假设	对设计的影响
----	-------	--------

编号	约束或假设	对设计的影响
C1	概念验证必须能在评阅人的机器上用一条命令运行，且无需配置	排除了服务端数据库、容器运行时以及前端构建步骤；直接导致选择 SQLite 与无框架客户端。
C2	平台不得涉及资金	排除支付、担保与退款，同时排除金融监管、纠纷处理与财务数据存储。
C3	项目无法接入学校身份认证系统	认证自成一体：用户名、密码，以及平台记录但无法核验的学号。
C4	项目周期为一个学年，团队为兼职投入	范围固定为单校区的交易闭环；推荐、通知与信誉体系明确后置。
C5	单校区，用户规模有限且有界	单节点数据库足够。多校区部署的容量规划不在范围内，列入后续工作。
A1	假设：学生有浏览器和校园网络	无需原生客户端；一套响应式 Web 客户端同时服务桌面与手机。
A2	假设：买卖双方可以在校园内当面交付	系统把交付建模为线下事件，不尝试安排或跟踪配送。
A3	假设：数据量绝对规模很小	除主键外没有索引是可接受的，第 6.10 节的扩展性测量也验证了这一点，但这一假设在「局限」一节中被重新审视。

2.6 如何验证工作

验证被当作过程的一部分，而不是收尾阶段的工作，通过三种持续运行的机制完成：一套 80 个用例的自动化测试，覆盖每个 JSON 接口，包括反向与边界路径，每次推送前运行；一套脚本化的界面检查，用真实浏览器驱动应用，在已登录与未登录两种状态下访问每个界面，一旦控制台报错或出现非预期的 4xx/5xx 响应就判定失败；以及一份需求追溯矩阵，从需求编号的那一刻起同步维护。矩阵见附录 B，结果在第 6 章报告。

3 需求与用例

3.1 需求建模方法

需求以编号的原子陈述记录，因为编号正是追溯的前提：无法被指代的需求，就无法被证明已经实现或测试。功能需求以 **FR** 为前缀，非功能需求 **NFR**，用例 **UC**，测试用例 **TC**。每条需求只含一个动词，描述系统做什么而不是怎么做，并且可被一个非通过即失败的测试所验证。

3.2 功能需求

表 3.1 给出 28 条功能需求，按所属功能分组；最后一列指明它在第 3.6 节哪个用例中被行使。

表 3.1 功能需求

编号	需求	分组	用例
FR-01	系统应允许访客使用用户名、密码以及可选的昵称、邮箱和学号注册账号。	账号	UC-01
FR-02	系统应依据用户名与密码认证已注册学生并建立会话。	账号	UC-02
FR-03	系统应允许已登录学生终止其会话。	账号	UC-02
FR-04	系统应返回已登录学生本人的资料，且不包含任何凭据信息。	账号	UC-12
FR-05	系统应允许已登录学生以标题、描述、价格、分类和成色发布商品。	发布	UC-06
FR-06	系统应允许为一件商品附加最多三张照片，每张以系统生成的文件名存储。	发布	UC-07
FR-07	系统应拒绝没有标题、没有价格，或价格非数字的发布请求。	发布	UC-06
FR-08	系统应允许商品的所有者修改其标题、描述、价格、分类和成色。	发布	UC-08
FR-09	系统应允许商品所有者将其下架，且不删除其历史记录与照片。	发布	UC-09
FR-10	系统应允许已下架商品的所有者重新上架。	发布	UC-09
FR-11	系统应以分页目录形式呈现商品，页大小固定并给出结果总数。	目录	UC-03
FR-12	系统应支持以关键词对标题与描述做大小写不敏感的搜索。	目录	UC-04
FR-13	系统应支持按分类筛选商品。	目录	UC-04
FR-14	系统应支持按状态筛选商品。	目录	UC-04
FR-15	系统应支持按发布时间、价格升序或价格降序排列商品。	目录	UC-04
FR-16	系统应呈现商品详情页，包含图廊、描述、价格、分类、成色、发布时间与卖家昵称。	详情	UC-05
FR-17	系统应按时间顺序呈现某件商品的公开留言。	详情	UC-10
FR-18	系统应允许已登录学生在商品下发表留言。	详情	UC-10
FR-19	系统应允许已登录学生将商品加入收藏。	收藏	UC-11
FR-20	系统应允许已登录学生取消收藏。	收藏	UC-11
FR-21	系统应列出已登录学生的收藏，最近收藏的在前。	收藏	UC-11
FR-22	系统应允许非卖家的已登录学生预订一件在售商品，并将其置为已预订。	交易	UC-13

编号	需求	分组	用例
FR-23	系统应允许已预订商品的卖家确认成交，并将其置为已售出。	交易	UC-14
FR-24	系统应允许已预订商品的卖家取消预订，将商品恢复为在售。	交易	UC-15
FR-25	系统应允许已登录学生向另一位注册学生发送私信。	私信	UC-16
FR-26	系统应按对方汇总呈现已登录学生的会话，并显示每个会话的最新一条消息。	私信	UC-17
FR-27	系统应允许已登录学生打开会话、按时间顺序阅读消息，并将收到的消息标记为已读。	私信	UC-18
FR-28	系统应提供个人中心，包含本人发布、收藏、已售出与会话。	个人中心	UC-12

3.3 非功能需求

表 3.2 非功能需求及其验证方式

编号	属性	需求	验证方式
NFR-01	易用性	从未用过本系统的学生，应在没有书面说明或培训的情况下完成一次发布。	对三名首次使用者的观察（第 6.11 节）
NFR-02	易用性	每个操作都应在界面上产生可见反馈；任何破坏性操作都不得在未经明确确认的情况下执行。	界面检查；对每个界面的脚本化抓取（第 6.11 节）
NFR-03	易用性	所有视图应共用同一套视觉语言：统一的字号级差、间距级差与组件样式。	对照组件样式表审查设计令牌
NFR-04	可访问性	每个交互控件都应可通过键盘访问与操作，并有可见的焦点指示；模态界面应向辅助技术暴露其角色与标题。	仅用键盘走查每个视图；检查 ARIA 属性
NFR-05	可访问性	从 430 像素的手机视口到 1440 像素的桌面视口，界面都应可用且不出现横向滚动。	两种视口的抓取（图 5.12）
NFR-06	性能	在 500 件商品的规模下，一次目录读取的应用层耗时不应超过 100 毫秒。	每个操作重复 200 次的测量（第 6.9 节）
NFR-07	性能	在 100 至 2000 条商品的区间内，目录读取成本应随数据量线性增长。	数据规模扩展测量（第 6.10 节）
NFR-08	可靠性	对任何通过界面或接口可达的输入，请求都不应产生未处理的服务端错误。	含畸形与缺失输入的 80 个自动化用例（第 6.3 节）
NFR-09	安全性	任何密码都不得以可还原形式存储或传输；凭据应以加盐哈希形式保存。	检查存储列；用例 TC-A10 断言
NFR-10	安全性	所有用户提供的文本在进入文档前都必须转义，使商品标题或留言无法在他人浏览器中执行脚本。	审查每一条渲染路径；以构造性输入测试
NFR-11	安全性	所有改变状态的操作都必须在服务端进行权限校验，不得依赖客户端隐藏控件。	权限矩阵（第 6.7 节）
NFR-12	可维护性与可移植性	在装有 Python 的干净 Windows 或 macOS 机器上，系统应以一条命令安装并启动，不需要服务器、容器或构建步骤。	在干净环境中安装所提交的代码（第 6.13 节）

3.4 需求优先级

需求使用 MoSCoW 方法排序，以便在进度紧张时能砍对东西。所用规则是：**必须**需求是那些一旦缺失就会破坏交易闭环的需求，其余都可以推迟而不至于让软件不再成立。

表 3.3 需求优先级

优先级	需求	理由
必须（18 条）	FR-01 至 FR-07、FR-09、FR-11 至 FR-16、FR-19、FR-22 至 FR-25、FR-28	缺少这些，学生无法发布、无法查找、也无法完成交易。任何一条缺失都会切断闭环。
应该（8 条）	FR-08、FR-10、FR-17、FR-18、FR-20、FR-21、FR-26、FR-27	没有它们系统仍可用，但体验明显变差：卖家无法修正商品信息，公开的可用性回答也消失了。
可以（2 条）	FR-13 与 FR-15 合并为一个带筛选的排序控件	是在既有能力上的细化，而非新增能力。
本项目不做	支付、配送、信誉、通知、推荐、多校区运营	每一项都在范围讨论中被否决。支付与配送带来法律与运营义务；信誉与推荐需要项目根本不会有的数据量。

3.5 系统上下文

图 3.1 把系统放回其环境中。边界之内是 Flask 应用以及它所提供的客户端代码；边界之外则是人类参与者、浏览器、保存数据库与照片的文件系统，以及安装系统的评阅人。上下文刻意做得窄：概念验证不依赖任何外部服务，这正是它能用一条命令装起来的原因。

系统上下文：PoC 与哪些人和系统交互

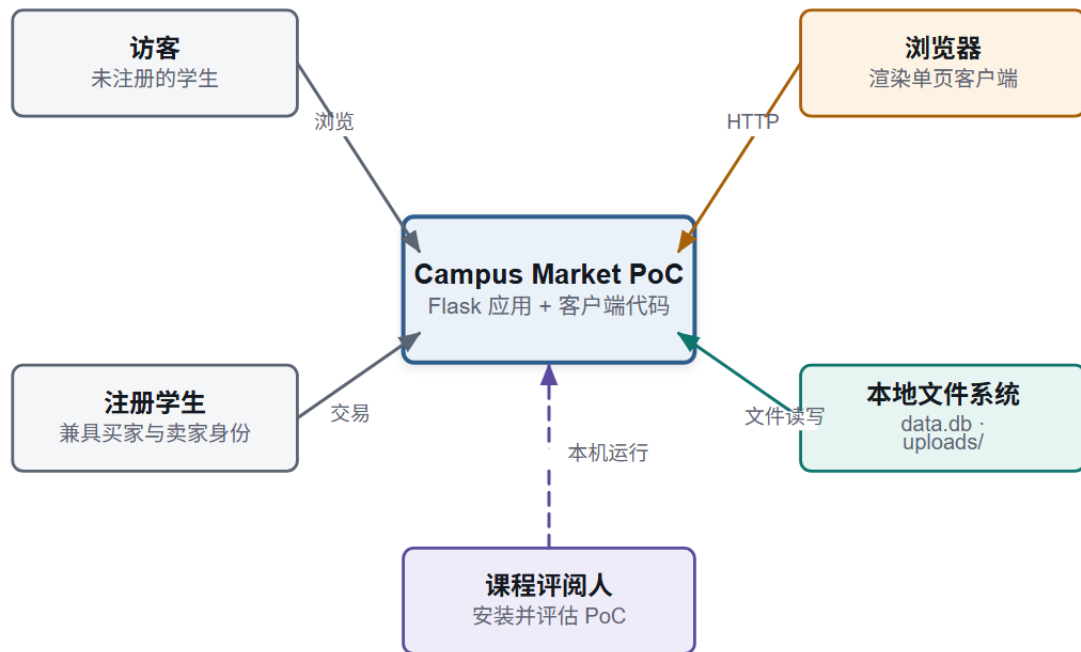


图 3.1 系统上下文：概念验证所交互的参与者、浏览器与本地文件系统

3.6 用例模型

图 3.2 给出 18 个用例及其两个发起者。两个参与者的区分在设计上有实际分量：未填充的用例无需会话即可访问，填充的用例则在客户端（把未登录访客重定向到登录视图）与服务端（独立拒绝同一请求）双重把关。访客参与者在泛化关系上是注册学生的父类，因此已登录学生继承访客可用的全部用例。

用例图：Campus Market

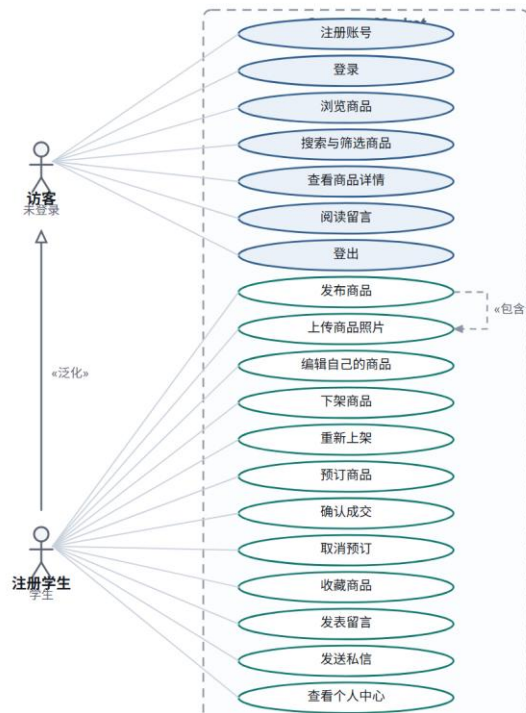


图 3.2 用例图：两个参与者下的 18 个用例，访客参与者泛化为注册学生

表 3.4 汇总这 18 个用例及各自满足的需求。其中工程风险最高的 8 个在第 3.7 节完整描述；其余 10 个结构相同，收录在附录 B 的需求追溯矩阵中。

表 3.4 用例汇总

编号	用例	参与者	需求
UC-01	注册账号	访客	FR-01
UC-02	登录与登出	注册学生	FR-02、FR-03
UC-03	浏览商品目录	访客	FR-11
UC-04	搜索、筛选与排序商品	访客	FR-12 至 FR-15
UC-05	查看商品详情	访客	FR-16
UC-06	发布商品	注册学生	FR-05、FR-07
UC-07	上传商品照片	注册学生	FR-06
UC-08	编辑自己的商品	注册学生	FR-08
UC-09	下架或重新上架商品	注册学生	FR-09、FR-10
UC-10	阅读与发表留言	访客、注册学生	FR-17、FR-18
UC-11	收藏与取消收藏	注册学生	FR-19 至 FR-21
UC-12	打开个人中心	注册学生	FR-04、FR-28
UC-13	预订商品	注册学生（买家）	FR-22
UC-14	确认成交	注册学生（卖家）	FR-23
UC-15	取消预订	注册学生（卖家）	FR-24
UC-16	发送私信	注册学生	FR-25
UC-17	查看会话列表	注册学生	FR-26
UC-18	打开并阅读会话	注册学生	FR-27

3.7 用例详细描述

以下 8 个用例给出完整描述，因为它们各自行使了无法从需求陈述推断出来的逻辑：双端校验、文件处理路径、带双重把关的状态机，以及一条正确性取决于「会话如何界定」的查询。

UC-06 发布商品

参与者	注册学生（卖家）
目标	学生有东西要卖，希望它出现在商品目录中。
前置条件	已存在登录会话，且学生位于「出售商品」视图。
主成功场景	1. 学生打开发布视图。 2. 系统呈现表单字段（必填项已标注），以及商品卡片的实时预览。 3. 学生填写标题、描述与价格，并选择分类和成色。 4. 学生选择 1 至 3 张照片。 5. 系统在浏览器端校验：标题非空、价格可解析为不小于零的数字、至少选择了一张照片。校验失败则就地提示并终止流程。 6. 系统上传照片并取得存储后的文件名。 7. 系统以「在售」状态创建商品并返回其标识。 8. 系统给出成功提示，清空表单，带学生回到目录页，新商品已出现在列表中。
替代与异常流程	5a. 某张照片超过 5 MB 或不是允许的图片类型：系统指出具体文件，且不将其加入。 5b. 选择的照片超过三张：系统拒绝多余部分并说明上限。 7a. 服务端校验拒绝该请求：系统展示服务端返回的原因，并保留已填写内容以免丢失。
满足的需求	FR-05、FR-06、FR-07

第 5 步与 7a 的校验是刻意重复的。浏览器负责即时反馈，服务端负责权威判定——因为请求完全可以不经过浏览器被构造出来。

UC-07 上传商品照片

参与者	注册学生（卖家）
目标	卖家需要把商品照片存下来，供商品条目引用。
前置条件	已存在登录会话，且卖家已选择一个或多个文件。
主成功场景	1. 系统接收所选文件。 2. 系统逐个检查文件扩展名是否在图片类型白名单内；只要有一个不合规，整体请求即被拒绝。 3. 系统从净化后的文件名中取出扩展名，并为每个文件生成一个随机唯一名。 4. 系统将文件写入上传目录。 5. 系统返回生成的文件名列表。 6. 卖家的客户端把这些文件名传给创建商品的请求，后者为每张照片记录一行。
替代与异常流程	1a. 请求中没有任何文件：系统返回校验错误。

	2a. 某文件的扩展名不在白名单内：系统返回错误并给出白名单内容。 3a. 净化后的文件名没有扩展名：该文件按不支持处理。
满足的需求	FR-06

生成随机文件名正是上传目录的防护手段：客户端提供的名字永远不会被当作路径使用。

UC-13 预订商品

参与者	注册学生（以买家身份）
目标	买家希望把商品留给自己，以免被他人买走。
前置条件	已存在登录会话，商品存在，且买家不是卖家。
主成功场景	1. 买家请求预订该商品。 2. 系统确认意图，并说明卖家会收到通知。 3. 系统查预订规则并读取商品当前记录。 4. 系统检查调用者与卖家不是同一人；若调用者就是卖家，请求被拒绝。 5. 系统检查存储的状态是否为「在售」；若为其他状态则拒绝，且不修改该行。 6. 系统写入新状态与修改时间。 7. 系统给出成功提示，详情页重新渲染为已预订状态，买家的操作被仅卖家可用的操作取代。
替代与异常流程	3a. 商品不存在：系统报告未找到。 4a. 买家试图预订自己发布的商品：拒绝。 5a. 商品已被预订或已售出：拒绝，并说明当前状态，让买家明白原因。 2a. 买家取消确认：不发送请求，什么都不变。
满足的需求	FR-22

两项检查彼此独立，且都不可省。角色检查回答「谁可以执行」，状态检查回答「何时可以」。只保留角色检查的设计，会允许一次预订覆盖已完成的成交。

UC-14 确认成交

参与者	注册学生（以卖家身份）
目标	卖家已把物品交付，希望将商品标记为已售出并关闭。
前置条件	已存在登录会话，且调用者是该商品的卖家。
主成功场景	1. 卖家请求将商品标记为已售出。 2. 系统确认意图。 3. 系统查确认规则、读取商品记录，并检查调用者是卖家、且状态为已预订。 4. 系统写入已售出状态与修改时间。 5. 系统给出成功提示；商品从在售目录中消失，并出现在卖家的已售出列表中。
替代与异常流程	3a. 状态不是已预订——例如卖家试图卖掉一件无人预订的商品：拒绝。 3b. 调用者不是卖家：以权限错误拒绝。 4a. 对已售出商品再做任何操作：拒绝，因为「已售出」没有出边。
满足的需求	FR-23

「只有从已预订状态才能确认成交」这条约束，正是状态字段可信的前提。没有它，卖家可以卖掉一件买家已经花力气预订的商品，而买家再也无法相信预订意味着什么。

UC-15 取消预订

参与者	注册学生（以卖家身份）
目标	买家没有完成交付，卖家希望商品重新可售。
前置条件	已存在登录会话，调用者是卖家，且状态为已预订。
主成功场景	1. 卖家请求取消预订。 2. 系统确认意图，并说明商品将重新回到在售状态。 3. 系统检查角色与当前状态。 4. 系统写入在售状态与修改时间。 5. 商品重新出现在目录中，任何买家都可以预订。
替代与异常流程	3a. 状态不是已预订：拒绝。 3b. 调用者不是卖家：拒绝。
满足的需求	FR-24

这是让预订这件事变得可用的恢复路径：一个永远无法释放的预订比没有预订更糟，因为一个不靠谱的买家就能让商品永久退出流通。

UC-11 收藏与取消收藏

参与者	注册学生
目标	学生希望记下一件正在考虑的商品，但还没有决定。
前置条件	已存在登录会话，且商品存在。
主成功场景	1. 学生请求收藏该商品。 2. 系统记录学生与商品之间的关联。 3. 系统给出提示，控件切换为已收藏状态。 4. 学生稍后请求取消收藏，系统删除该关联。 5. 学生的收藏列表（最近收藏在前）可在个人中心查看。
替代与异常流程	1a. 该商品已被收藏：系统拒绝重复记录而不是写入第二行，并提示已收藏。 1b. 不存在会话：请求被拒绝。
满足的需求	FR-19、FR-20、FR-21

唯一性由数据库约束保证，而不是由应用层「先读再写」的检查保证——后者在读写之间存在竞态，唯一约束不存在这个问题。

UC-18 打开并阅读会话

参与者	注册学生
目标	学生希望就某件商品继续沟通。
前置条件	已存在登录会话，与对方的会话已经存在或正在开始。

主成功场景	1. 学生打开与另一位学生的会话。 2. 系统取出两人之间双向的全部消息，按时间排序。 3. 系统把学生收到的、来自对方的消息标记为已读。 4. 系统以气泡形式呈现消息，区分本人与对方，并滚动到最新一条。 5. 学生输入回复并发送。 6. 系统记录消息，并回到第 2 步重新渲染会话。
替代与异常流程	1a. 会话不存在：系统呈现空会话并邀请发起。 5a. 学生发送空消息：系统忽略该次提交。 5b. 消息无法存储：系统报告失败并保留学生已输入的内容。
满足的需求	FR-27

把「标记已读」作为「阅读」的一部分而不是单独一次调用，消除了客户端忘记上报已读的一整类缺陷。

3.8 业务流程模型

图 3.3 为软件所支持的过程建模，覆盖从发布到交付的全过程。三条泳道把两种人类角色与系统分开，这使交接点变得可见：交易只有两次离开买家泳道，一次是预订必须到达卖家，一次是付款与交付在线下发生。最后一条泳道承载守卫条件，因为守卫正是系统对这个过程的贡献——没有它们，这个过程就和那个已经在失败的线下过程完全相同。

业务流程：一笔完整的二手交易

每个角色一条泳道，时间从左向右；虚线表示跨泳道传递。

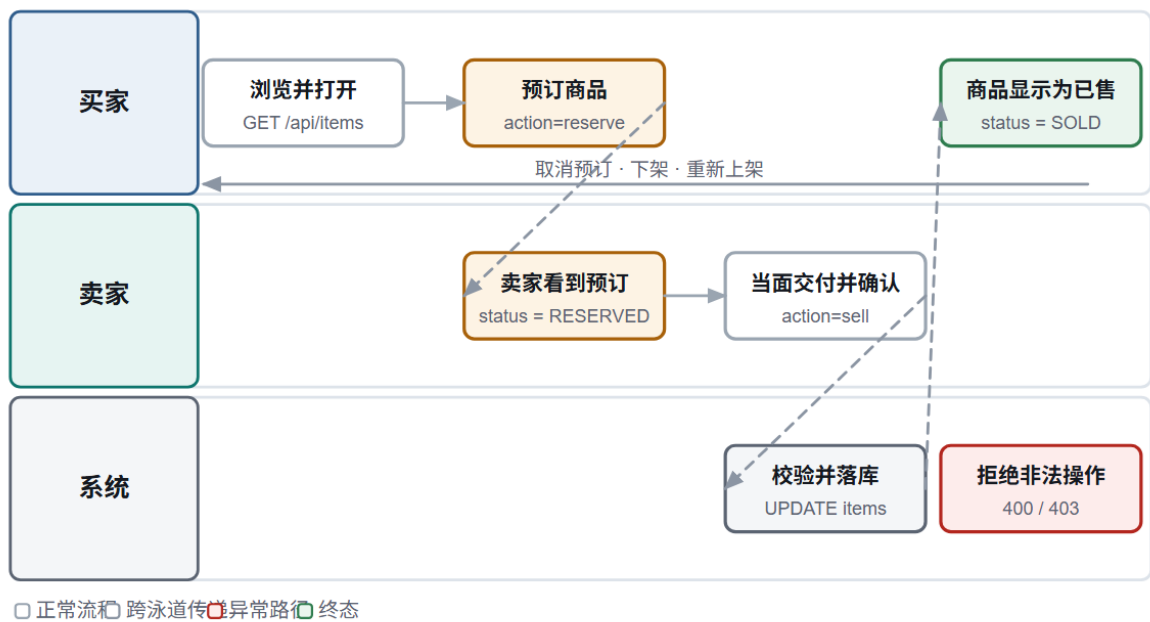


图 3.3 一笔完整二手交易的业务流程模型，服务端守卫显示在系统泳道中

3.9 发布活动的活动模型

图 3.4 用活动图对发布建模，因为这是系统中唯一一个真正存在依赖序列的过程。图中有三处判断，而每一处都被检查两次：一次在浏览器端用于即时反馈，一次在服务端因为浏览器不可信。该图也直观说明了为什么「创建商品」与「上传照片」是两个独立请求：上传在创建商品之前完成，因此第二步失败不可能留下一条引用了不存在的照片的商品记录。

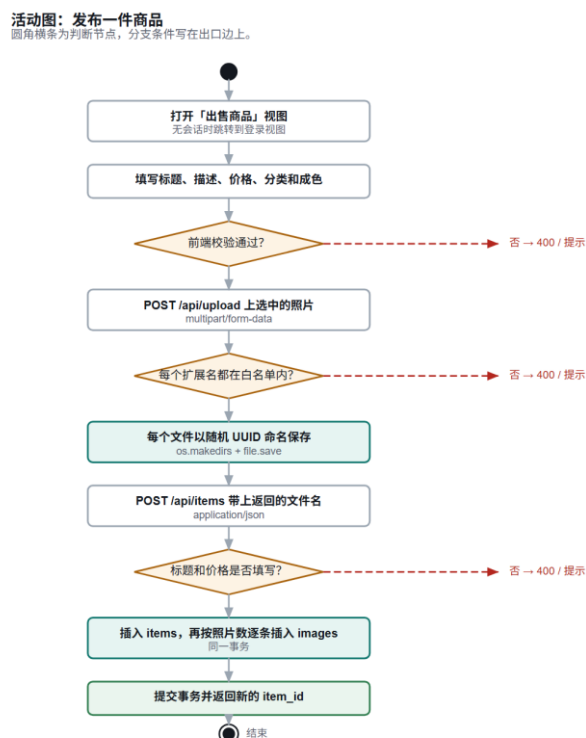


图 3.4 发布商品的活动模型，含三处判断与前后端重复执行

3.10 需求追溯

追溯在两个方向上都保持。向前，每条需求都指明行使它的用例，每个用例都指明实现它的设计要素；向后，每个测试用例都指明它所验证的需求，因此一个失败的测试不仅指出哪里坏了，也指出哪条需求现在有风险。

完整矩阵见附录 B。第 6.8 节汇总覆盖情况，并明确指出哪些需求的验证最弱。矩阵是在需求编号的那一刻起同步维护的，而不是最后拼凑的；事后重建的矩阵几乎总会在某条需求被悄悄丢掉的地方出现缺口。

4 系统设计与架构

4.1 架构风格与被考虑的替代方案

系统是一个三层分层的 Web 应用：运行在浏览器中、不含框架的客户端；掌握全部逻辑与全部权限判定的 Flask 应用；以及由单个 SQLite 文件和上传目录构成的薄持久层。每一层只与下一层通信，客户端从不直接访问数据库。

该风格是针对三个替代方案比较后选定的，三次判断中的决定性因素都是第 2.5 节的约束 C1——软件必须能在评阅人的机器上用一条命令运行。

表 4.1 架构替代方案及各自的否决理由

替代方案	吸引力	否决理由
三层分层（已采用）	表示与逻辑分离、逻辑与存储分离；每层可独立修改；整套系统以单进程运行，无外部依赖	采用。它在满足 C1 的同时仍给出清晰的关注点分离，并为权限校验提供了明确的位置。
服务端渲染页面	比客户端应用更简单：没有 JSON 契约，也不需要重复校验	否决，因为界面需要在不整页刷新的情况下更新——留言区、收藏控件和聊天面板都是就地变化的，服务端渲染意味着每一次都要重载页面。
前端分离 + 独立 API 服务	关注点分离最彻底，也是生产环境中最常见形态	否决，因为它需要第二个进程或跨域配置，两者都会破坏 C1；跨域请求还需显式携带会话 Cookie。
微服务	账号、商品与私信可独立部署	否决，因为不成比例。整个应用约 1,700 行；拆分只会为系统并不存在的扩展问题引入服务发现、服务间契约和分布式故障模式。

4.2 分层架构

图 4.1 给出三个层次及各自职责。最关键的是表示层与应用层之间的边界：客户端不掌握任何权威。所有保护数据的规则——谁能改什么、状态允许变成什么、哪些文件类型可以保存——都在边界之下执行。客户端了解这些规则只是为了给出更好的反馈，设计上也接受把部分校验写两遍的代价，以换取一个无法通过手工构造请求绕过的系统。

分层架构与各层职责

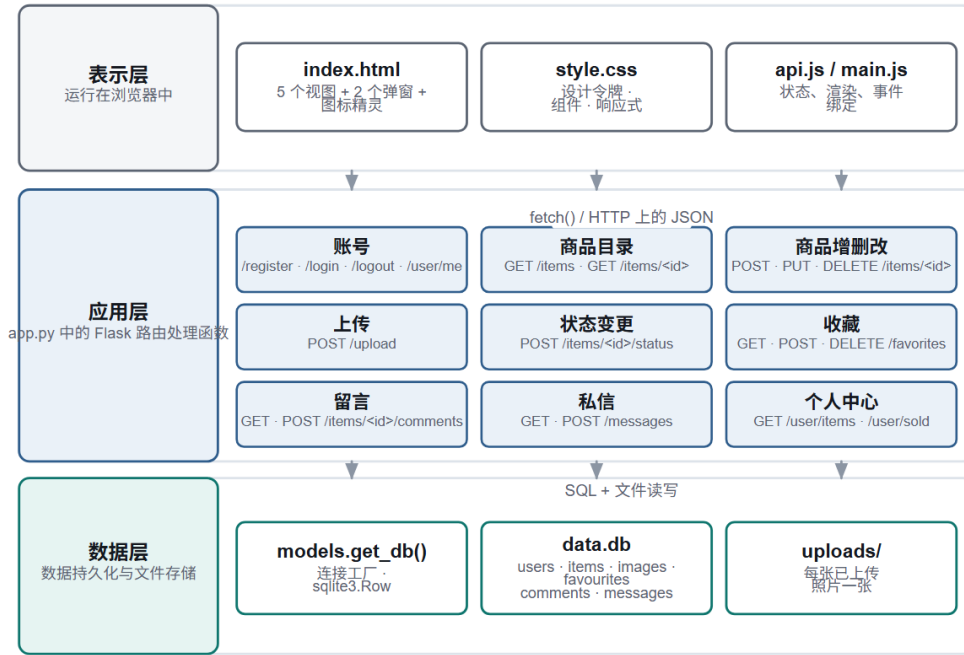


图 4.1 分层架构：应用层的接口分组与数据层的组件

这一安排中有两点承载了最多设计分量。其一，表示层从不直接访问数据库——每一次读和每一次写都经由 JSON 接口，这意味着权限校验只需要在一处正确。其二，数据层刻意做薄：它只知道表和连接，不知道任何规则。这种「薄」正是将来迁移到托管数据库可以成为局部改动而非重写的原因，也是即便概念验证并不需要可移植性仍坚持做薄的理由。

表 4.2 各层职责，以及各层被禁止做的事

层次	负责	不得
表示层	视图状态、渲染、事件处理、即时输入校验，以及在插入文档前转义全部用户提供的文本	掌握任何数据的权威；拼写 SQL；成为某条规则唯一的执行点
应用层	HTTP 路由、会话处理、权限校验、输入校验、状态机、构造 JSON 响应信封、文件存储	包含表示层标记；假设请求一定来自所提供的这套客户端
数据层	创建连接、关系模式、唯一性约束、磁盘文件存储	包含应用层同样需要知道的业务规则

4.3 模块划分

图 4.2 给出各模块及依赖方向。划分遵循 Parnas 提出的准则——隐藏最可能变化的部分：数据模型形态藏在数据层之后，HTTP 契约形态藏在客户端的请求封装之后，密码哈希与文件名净化的细节则藏在第三方库之后。

模块划分与依赖方向

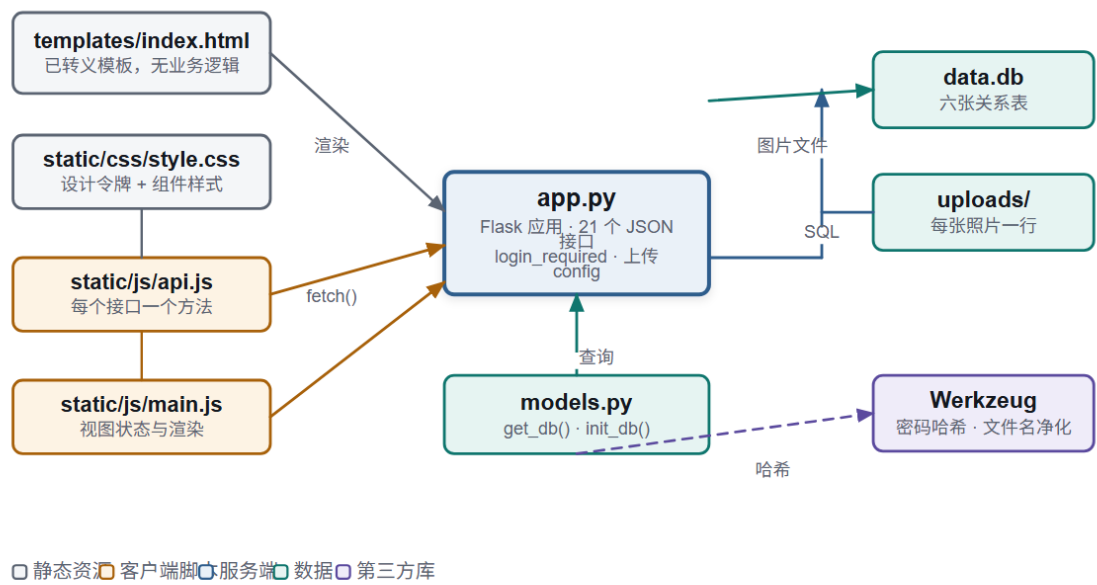


图 4.2 模块划分与依赖方向

表 4.3 模块清单

模块	行数	职责	依赖
app.py	552	Flask 应用对象与全部 23 条路由：账号、目录、商品管理、上传、交易、收藏、留言、私信与个人中心	models.py、Werkzeug
models.py	106	连接工厂与六张表的模式创建	sqlite3
templates/index.html	441	唯一的文档：五个视图容器、两个模态界面、内联 SVG 图标精灵，以及客户端渲染的挂载点	style.css、api.js、main.js
static/js/api.js	63	每个接口一个方法，URL 不会被写两遍，请求形态只在一处定义	HTTP 接口本身
static/js/main.js	1169	全部视图状态、渲染、事件绑定、hash 路由、转义与反馈层	api.js
static/css/style.css	1025	设计令牌、组件样式、响应式断点	无

4.4 领域模型

图 4.3 给出领域模型。之所以称其为「面向持久化」，是因为系统没有使用对象关系映射：图中的类被实现为数据表以及查询返回的字典，而不是 Python 类。即便如此建模仍然值得，因为关联的基数关系正是模式必须保证的东西。

领域类图（面向持久化）

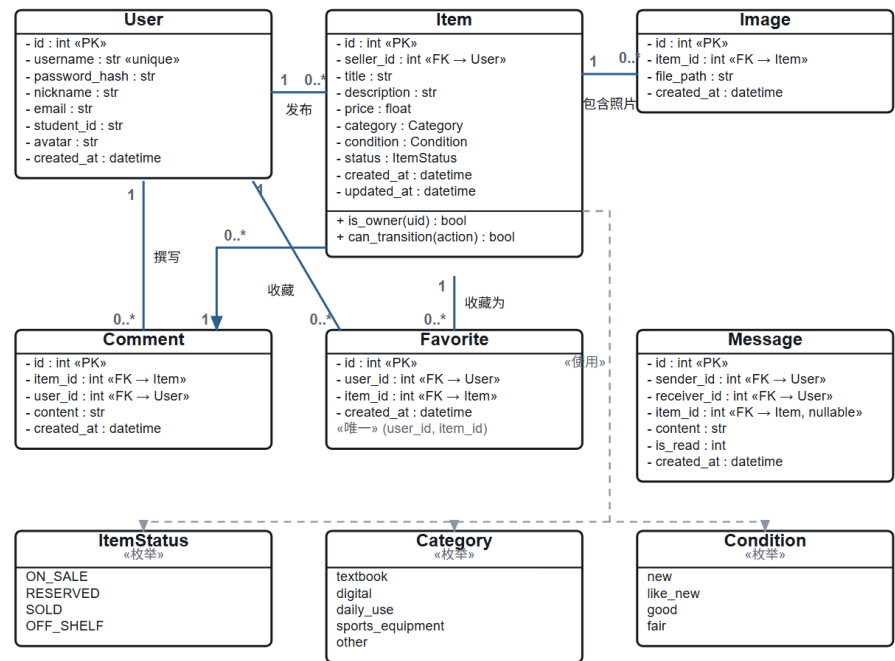


图 4.3 领域类图：六个实体及其属性与关联

4.5 数据库设计

模式包含六张表，见图 4.4，已满足第三范式：每个非键属性都依赖于完整的键，与其他属性无关。设计时对照范式做了检查，其中一处候选反范式化——把封面照片的引用直接存在商品行上——被否决，改用以 `images` 表为目标的关联子查询，因为子查询显然正确，而冗余列需要额外代码保持同步。

SQLite 数据库的实体关系图

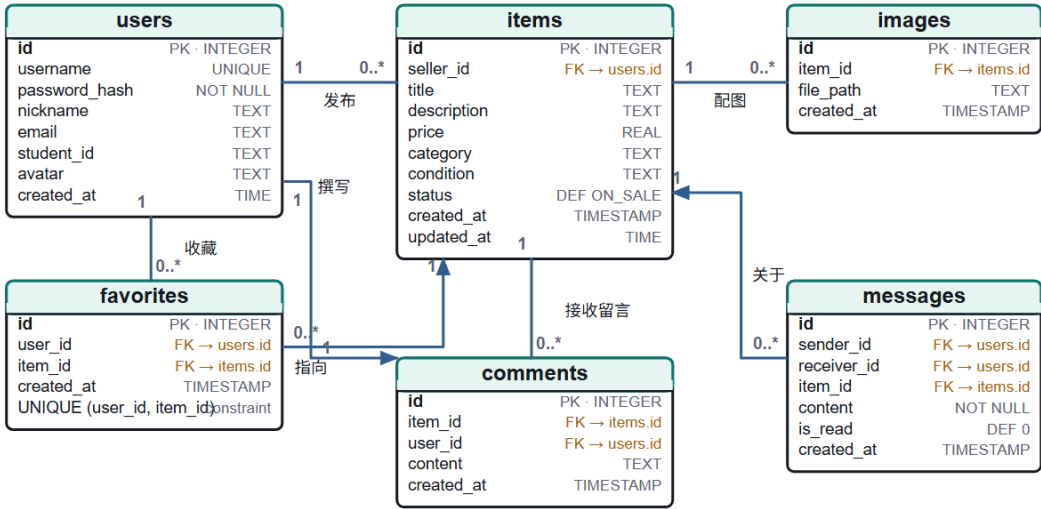


图 4.4 六张表的实体关系图，含外键与基数

表 4.4 数据表规格

表	用途	主键	约束与说明
users	每个注册学生一行	id	username 唯一且非空; password_hash 非空; nickname、email、student_id、avatar 可选
items	每个商品一行，聚合根	id	seller_id 外键指向 users; title 非空; price 非空且为实数; status 默认在售; 每次改动都写入 updated_at
images	商品的一张照片一行	id	item_id 外键指向 items; file_path 非空; 存储的是生成的文件名，从不使用客户端提供的名字
favorites	学生与收藏商品之间的多对多关联	id	user_id 与 item_id 均为外键; UNIQUE(user_id, item_id) 阻止重复收藏
comments	挂在商品下的公开提问或回答	id	item_id 与 user_id 均为外键; content 非空
messages	两名学生之间的私信，可关联某件商品	id	sender_id 与 receiver_id 外键指向 users; item_id 外键指向 items 且可为空; is_read 默认零

SQLite 的两个特性影响了设计，两者都作为局限而非疏漏记录在案。其一，只有当连接以 `PRAGMA foreign_keys = ON` 打开时引擎才会强制所声明外键，而概念验证并未设置该 `pragma`，因此当前的引用完整性是由应用层维持的约定。其二，`CURRENT_TIMESTAMP` 只精确到秒，任何仅依赖它排序的场景在两条记录写入同一秒时都不稳定——这个缺陷由测试套件发现，并在第 6.13 节报告。

4.6 交易状态机

商品的状态字段被建模为显式的状态机，而不是一个界面碰巧会设置的自由文本字段。图 4.5 给出四个状态与五条迁移，表 4.5 给出规则。这个模型是项目最重要的设计决策，因为它让「这件还在吗」的答案从参考意见变成了权威判定。

商品状态机

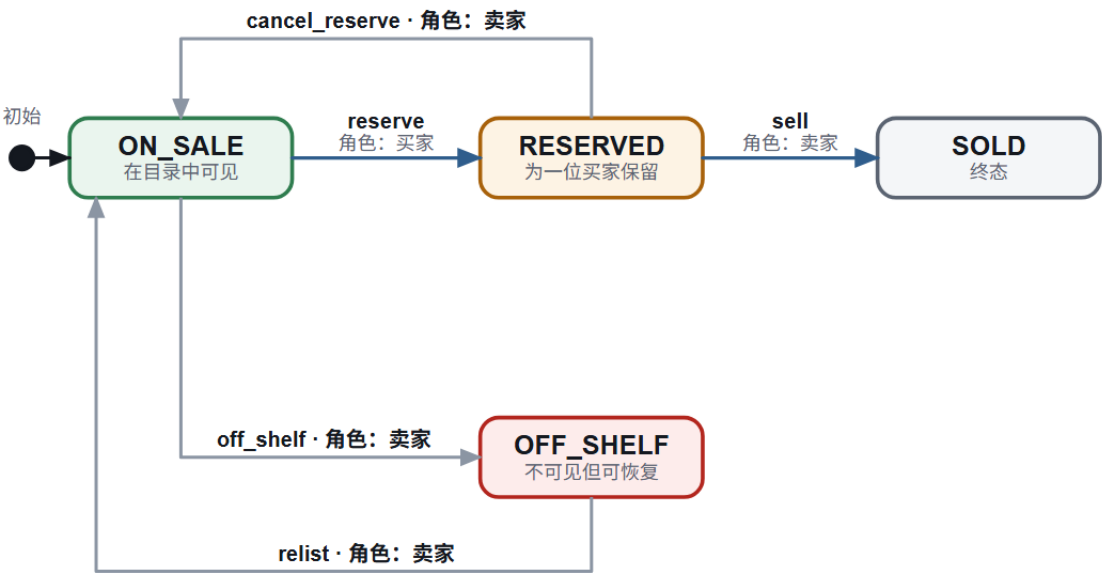


图 4.5 商品状态机：每条迁移的参与者与守卫条件

表 4.5 状态迁移规则。角色与原状态两者都必须满足；任一不满足的请求不改变任何数据。

操作	原状态	新状态	允许的参与者	失败响应
reserve	在售	已预订	除卖家以外的任何学生	调用者是卖家时 403；状态不是在售时 400
sell	已预订	已售出	卖家	调用者不是卖家时 403；状态不是已预订时 400
cancel_reserve	已预订	在售	卖家	调用者不是卖家时 403；状态不是已预订时 400
off_shelf	在售	已下架	卖家	调用者不是卖家时 403；状态不是在售时 400
relist	已下架	在售	卖家	调用者不是卖家时 403；状态不是已下架时 400
（其他）	—	—	—	在读取商品之前即返回 400 非法操作

「已售出」没有出边，这是刻意选择而非遗漏：撤销一次已完成的成交，需要追踪是哪位买家以什么方式完成的，而概念验证根本没有建模成交后的买家侧。其后果是误操作无法通过界面撤销，这也正是该操作的确认弹窗会明确写出「商品将被标记为已售出」的原因。

4.7 交互设计

有三处交互承载了系统的大部分行为，因此完整建模。它们合起来行使了每一层、状态机的两个方向，以及系统中唯一一条正确性不能靠阅读直接看出的查询。

4.7.1 发布商品

图 4.6 跟踪一次发布从表单提交到写入数据行的全过程。该时序图明确了两项设计决策。上传与创建是两个独立请求，上传先完成并返回文件名，之后才写入任何商品行；因此第二步失败时不可能留下一条引用了不存在照片的商品记录。而商品行先于照片行写入，使生成的主键可直接用作照片行的外键——反过来则需要再读一次。

时序图：从提交表单到写入数据库的发布流程
实线箭头为调用，虚线为返回；自调用为一个组件内部的处理。

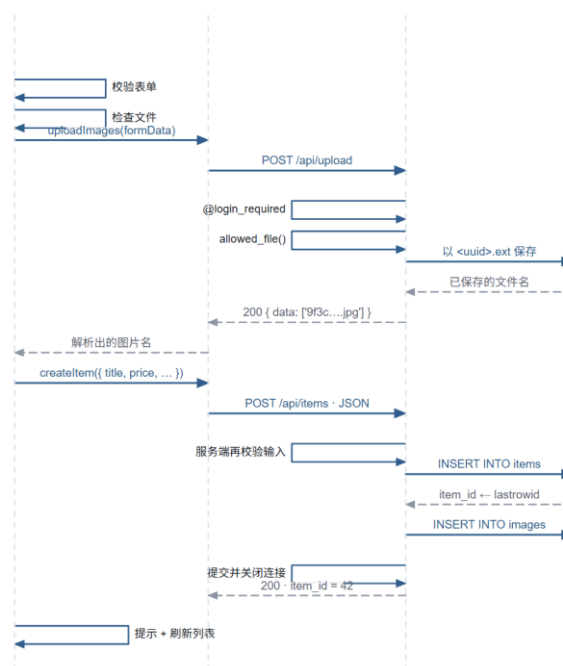


图 4.6 发布商品的时序图：从提交表单到写入数据行

4.7.2 预订商品与确认成交

图 4.7 给出交易的两端。两个请求到达同一个接口，并按同一张规则表校验，这正是状态机可信的原因：两条迁移不可能各自走偏，因为它们只有一处定义。图中也显式画出了失败路径，因为「被拒绝的请求不改变任何数据」属于规格的一部分，而不是实现细节。

时序图：预订商品与确认成交
两次状态变更在写库前都按同一张规则表校验。

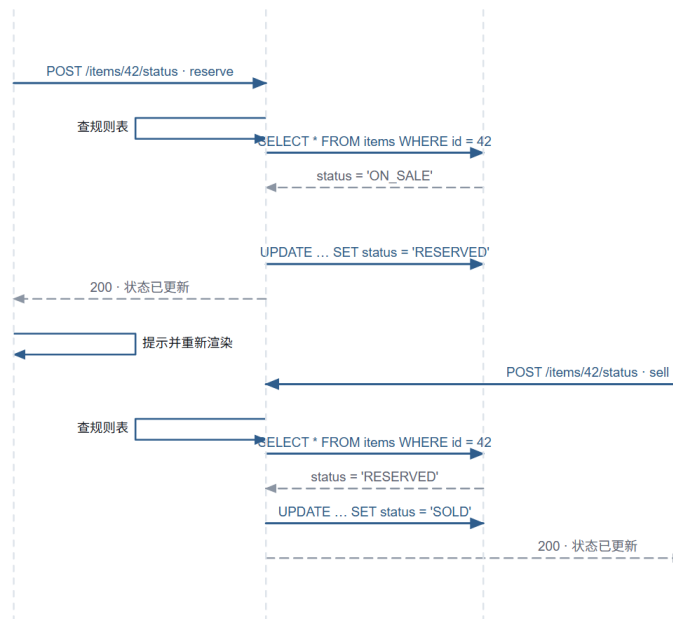


图 4.7 预订商品与确认成交的时序图，含两处守卫与拒绝路径

4.7.3 会话与私信

图 4.8 覆盖三处私信交互。会话列表由一条聚合查询产生，而不是每个会话查一次；这一点很重要，因为一名学生的会话列表会无限增长，而与他通信的人数并不会。打开会话同时包含读与写：消息取出与收到消息标记已读发生在同一次请求里，因此未读状态不可能与学生实际看到的内容脱节。

时序图：会话列表、打开会话与回复
会话列表由一条聚合查询产生，而不是每个会话查一次。

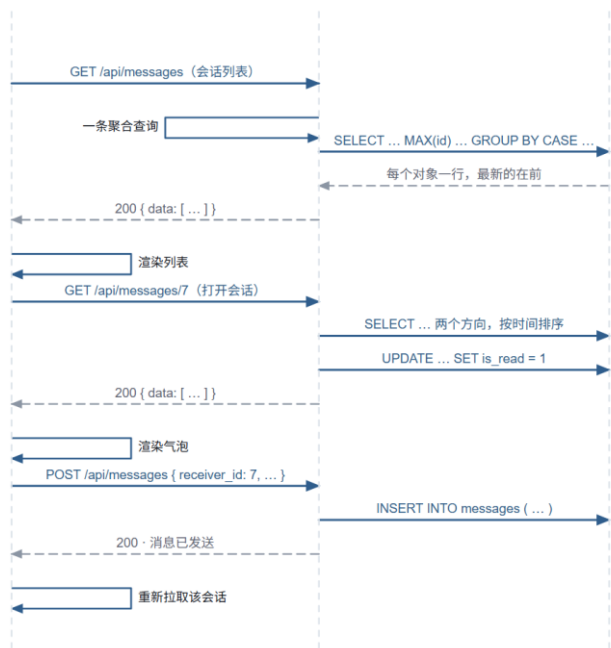


图 4.8 会话列表、打开会话与回复的时序图

4.8 接口设计

客户端通过一组小型 JSON 接口与服务端交互。图 4.9 按作用资源对接口分组，表 4.6 列出各接口接受的方法及是否需要会话。所有接口都使用表 4.7 描述的同一种响应信封，使客户端无需检查响应体形态即可统一处理成功与失败。

JSON 接口一览（按资源分组）

共 23 条路由：2 条页面路由、21 个 JSON 接口。带底色的方法名表示需要登录会话。

账号	POST	/api/register	公开
	POST	/api/login	公开
	POST	/api/logout	公开
	GET	/api/user/me	需会话
商品目录	GET	/api/items	公开
	GET	/api/items/<id>	公开
商品管理	POST	/api/items	需会话
	PUT	/api/items/<id>	需会话
	DELETE	/api/items/<id>	需会话
媒体	POST	/api/upload	需会话
	GET	/uploads/<filename>	公开
交易	POST	/api/items/<id>/status	需会话
社交	GET	/api/favorites	需会话
	POST	/api/favorites	需会话
	DELETE	/api/favorites/<id>	需会话
	GET	/api/items/<id>/comments	公开
	POST	/api/items/<id>/comments	需会话
	GET	/api/messages	需会话
	POST	/api/messages	需会话
	GET	/api/messages/<uid>	需会话
个人中心	GET	/api/user/items	需会话
	GET	/api/user/sold	需会话

图 4.9 按资源分组的 JSON 接口一览，含方法与会话要求

表 4.6 接口清单。标记「需要」表示未登录时直接返回 401，且不解析请求体。

方法	路径	用途	会话
POST	/api/register	创建账号	不需要
POST	/api/login	认证并建立会话	不需要
POST	/api/logout	终止会话	不需要
GET	/api/user/me	返回调用者本人资料	需要
GET	/api/items	目录，支持关键词、分类、状态、排序与分页	不需要
GET	/api/items/<id>	商品详情，含图片与留言	不需要
POST	/api/items	创建商品	需要
PUT	/api/items/<id>	修改调用者拥有的商品	需要
DELETE	/api/items/<id>	下架商品（软删除）	需要
E			
POST	/api/upload	存储一张或多张照片并返回文件名	需要
GET	/uploads/<name>	提供已存储的照片	不需要
POST	/api/items/<id>/status	执行一次状态迁移	需要
GET	/api/favorites	列出调用者的收藏	需要
POST	/api/favorites	收藏商品	需要
DELETE	/api/favorites/<id>	取消收藏	需要
E			
GET	/api/items/<id>/comments	读取商品的公开留言	不需要
POST	/api/items/<id>/comments	发表留言	需要
GET	/api/messages	按对象汇总的会话列表	需要
POST	/api/messages	发送私信	需要
GET	/api/messages/<uid>	读取会话并标记已读	需要

方法	路径	用途	会话
GET	/api/user/items	调用者本人的发布	需要
GET	/api/user/sold	调用者已售出的商品	需要

表 4.7 所有接口共用的响应信封

情形	HTTP 状态	响应体	客户端行为
成功且带数据	200	{ code: 200, data: ... }	渲染数据
成功但不带数据	200	{ code: 200, msg: '...' }	以提示形式展示消息
校验失败	400	{ code: 400, msg: '<原因>' }	在出错的控件旁展示原因
未认证	401	{ code: 401, msg: '...' }	跳转到登录视图并记住原意图
无权限	403	{ code: 403, msg: '<原因>' }	展示原因，不重试
未找到	404	{ code: 404, msg: '...' }	展示空状态并提供返回路径
请求体过大	413	框架返回的 HTML 错误页	提示文件超过 5 MB 上限

4.9 客户端设计

客户端是单个文档，包含五个视图容器，由一个小型路由控制显示与隐藏，见图 4.10。有三项设计决策值得说明。其一，**hash 路由 #item-<id>** 让单件商品可以被寻址，因此商品链接可以粘贴到消息中并直接打开详情页；该路由在页面加载时解析，并在每次 hash 变化时重新解析。其二，**守卫是重复的**：客户端把未登录访客从发布和个人视图重定向出去，服务端则独立拒绝同一请求。其三，**隐藏的容器不会被重建**：切换视图只重新渲染发生变化的部分，因此写到一半的商品表单在切到目录再回来时仍保留内容。

前端视图路由

单个 HTML 文档、五个视图，以及让单件商品可分享的 hash 路由。

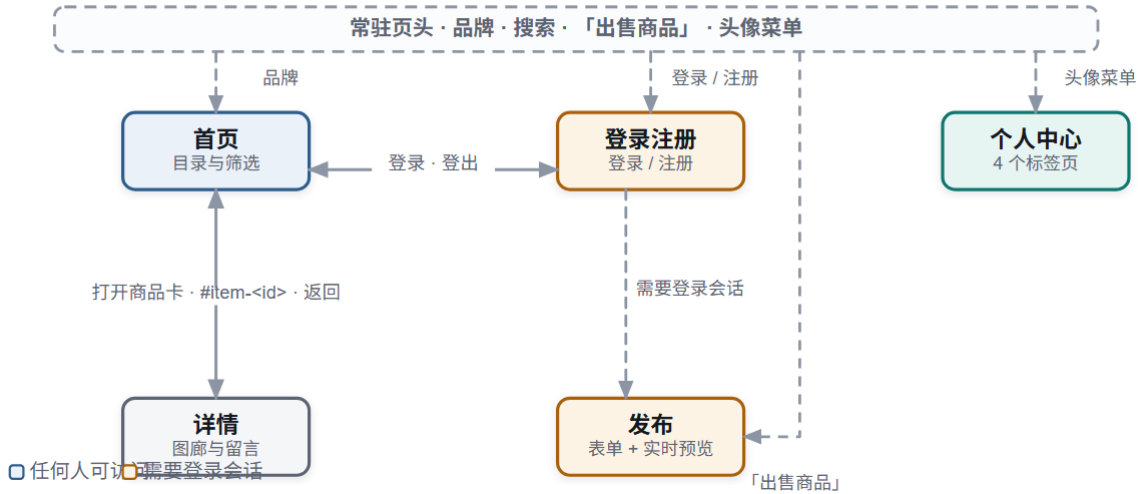


图 4.10 前端视图路由：五个视图、常驻页头，以及需要会话的视图上的守卫

4.10 设计决策与取舍

表 4.8 记录塑造了系统的各项决策、各自的替代方案，以及明知代价而接受的取舍。记录被否决的替代方案很重要：一份只列结果的决策日志无法被评审，因为无从判断替代方案是被考虑后否决，还是根本没被想到。

表 4.8 设计决策日志

编号	决策	被否决的替代方案	接受的代价
D1	由单个 Flask 进程同时提供客户端与接口	独立的 API 服务或第二个进程	客户端与服务端无法独立部署或扩展。接受，因为 C1 禁止第二个进程。
D2	以 SQLite 作为存储	PostgreSQL 或托管关系数据库服务	没有并发写入能力，其他客户端无法经网络访问，外键默认不强制。接受，因为模式得以随作业一起提交，且无需安装任何东西。
D3	不使用对象关系映射	SQLAlchemy 或同类库	29 条 SQL 全部手写，模式形态直接出现在路由处理函数中。接受，因为查询集合很小，且第 5.7 节的聚合查询直接书写比经过映射器更清晰。
D4	浏览器与服务端重复校验	只在服务端校验	部分规则写了两遍，两份可能走偏。接受，因为只在客户端校验等于没有校验，而只在服务端校验会让用户在得到任何反馈前先等一个来回。
D5	用规则表驱动状态迁移	为每个操作写一串条件语句	合法迁移以数据结构描述而非控制流表达，有人会觉得不够直观。接受，因为规则集可以在一处读完，且不可能与实际执行的不一致。
D6	商品采用软删除	删除数据行及其照片	已下架商品仍留在数据库中，其照片永不回收。接受，因为卖家的历史与买家的会话因此保持自治，误删除也可恢复。
D7	上传文件使用生成的文件名	沿用客户端提供的文件名	磁盘上的文件无法通过文件名识别。接受，因为客户端提供的名字既是路径穿越风险，也可能与既有文件冲突。
D8	每次状态迁移都要通过两处独立守卫	只做角色检查	每个请求多一次读取，也多一种失败模式要测。接受，因为角色检查回答「谁」，状态检查回答「何时」，而一次预订覆盖已完成的成交是正确性故障。

编号	决策	被否决的替代方案	接受的代价
D9	会话列表使用单条聚合查询	每个会话查一次，或缓存列表	该查询比简单查询更难读。接受，因为它使成本不随会话数量增长，而替代方案会为列表的每一行各查一次。
D10	不提供支付能力	接入支付服务商	平台无法抽佣，交付必须由两名学生自行约定。刻意接受：支付会把金融监管、纠纷处理与欺诈防范带进一个课程项目。

5 实现

5.1 技术栈

概念验证用 Python 与 Flask 编写，使用 SQLite 存储，并提供一套由纯 HTML、CSS 与 JavaScript 构成、不含框架也没有构建步骤的客户端。图 5.1 给出每项选择的理由；所有组件都用同一条标准筛选——评阅人能否在不买账号、不装容器、不做编译的前提下把它跑起来？

技术选型与理由

所有组件的选择标准只有一个：让 PoC 能在一台笔记本上用一条命令装起来。

Python 3.8+ 实现语言	机房和教师机上已装好，无需额外采购运行时
Flask 2.3.3 Web 框架	路由、会话与 JSON 处理，只引入一个小依赖
Werkzeug 2.3.7 安全工具	密码哈希与文件名净化，不用再加库
SQLite 3 数据库	文件型，无需安装服务端；数据库随作业一起提交
原生 JS / CSS 客户端	无构建步骤、无 node_modules；由 Flask 直接打开 index.html
Playwright + Edge 验证工具	用于抓取本报告中的界面状态

图 5.1 技术选型与理由

表 5.1 组件版本与作用

组件	版本	在系统中的角色
Python	3.8 及以上（开发环境为 3.13）	应用与测试套件的运行时
Flask	2.3.3	路由、请求解析、签名会话与 JSON 响应
Werkzeug	2.3.7	密码哈希与文件名净化
SQLite	3（随 Python 提供）	关系存储，磁盘上单个文件
pytest	9.x	第 6 章所述测试套件的运行器
Playwright（Node）	playwright-core	驱动 Microsoft Edge 完成界面验证与本报告中的界面抓取

5.2 软件结构与规模

表 5.2 给出所交付软件各部分的规模。分布本身值得注意：客户端比服务端更长，这也是一个复杂度落在交互而非计算上的系统的预期形态。服务端以 658 行实现 21 个 JSON 接口，平均每个接口 31 行，其中已包含校验与权限判定。

表 5.2 所交付软件各部分的规模

组成部分	行数	占比
static/js/main.js	1,169	34%
static/css/style.css	1,025	30%
app.py	552	16%
templates/index.html	441	13%
models.py	106	3%
static/js/api.js	63	2%
tests/（测试套件与测量脚本）	约 1,300	不计入运行时
运行时合计	3,356	100%

5.3 账号、会话与凭据存储

注册接受用户名、密码与三个可选资料字段。密码从不存储：它经过带盐的密钥派生函数处理，只把结果哈希写入数据库。实现刻意使用该库的默认参数而不自行选择，遵循的原则是自研密钥派生函数本身就是负债。有一个测试断言存储值既不包含提交的密码，又带有预期的算法前缀，因此将来若有人误存明文，测试会失败而不是无声通过。

会话保存在 Flask 的签名会话 Cookie 中。应用对它的唯一操作是在登录时写入已认证用户标识、在登出时将其移除。一个装饰器被施加到每个受保护路由上，读取该标识并在缺失时以 401 拒绝请求。由于装饰器施加在路由层面而不是在函数内部判断，新增接口若忘记加它会在评审中立刻显现——而第 6.7 节的权限矩阵正是为了让这种失败模式不可能被忽略。

5.4 商品目录查询

目录接口接受五个可选参数——关键词、分类、状态、排序与页码——并据此构造一条语句。构造方式是把条件片段追加到列表、并同步扩展一个平行的绑定参数列表，绝不把取值插进语句文本。这一区别很关键：拼接关键词会让搜索框成为注入点，而绑定不会。同一份条件列表还被复用来构造计数查询，因此驱动分页控件的总数与它所描述的那一页用的是完全相同的筛选条件。

关键词在两个列上做两侧通配匹配，这正是搜索「laptop」能找到「Lenovo laptop bag」的原因。每件商品的封面照片用以 images 表为目标的关联子查询取得，而不是用连接，因为连接会让商品行按照片数量重复，而目录页每张卡片只需要一张图。两侧通配的

代价是全表扫描，因为前置通配无法使用索引；在概念验证的规模下这无关紧要，第 6.10 节对此有测量；在更大规模上，答案是全文索引，而不是修改这条查询。

5.5 照片上传与存储

上传是系统中唯一触及文件系统的部分，因此被严肃对待。接口接收包含一个或多个文件的 `multipart` 请求，逐个检查扩展名是否在白名单内，从净化后的文件名中取出扩展名，然后以一个由随机通用唯一标识生成的名字写入文件。客户端提供的名字除了用来读取扩展名之外不被用于任何事。

表 5.3 上传与发布所应用的校验规则及各自的执行位置

规则	限制	浏览器端执行	服务端执行
接受的文件类型	png、jpg、jpeg、gif、webp	按 MIME 类型在选择时校验	按扩展名对照白名单校验
单个文件大小上限	5 MB	选择时校验	由框架的请求体大小限制执行（413）
每件商品的照片数上限	3	选择时校验	未执行；该限制只在客户端生效
标题必填	非空	提交前校验	写入数据行前校验
价格必填且为数字	不小于零	提交前校验	写入数据行前校验
至少一张照片	1	提交前校验	未执行；商品也可以合理地不带照片

表 5.3 的最后两行是如实记录的空缺。三张照片上限与「至少一张照片」只存在于客户端。手工构造的请求可以发布一件没有照片、或带三十张照片的商品。两处遗漏都不会破坏数据，界面也不会产生这样的请求，因此概念验证接受了这一风险，并将其列为待修复缺陷，而不是当作对 FR-06 的完整实现来呈现。

5.6 状态机

状态机实现为一个字典，把每个操作映射到它要求的原状态、结果状态以及被允许执行的角色。接口随后对每个操作执行同样的五步，见图 5.2：拒绝未知操作、读取商品并拒绝不存在的商品、检查角色、检查原状态，最后才写入。由于这五步是共用的，新增一个状态只需在表里加一项、在界面加一个控件，而无需改动校验逻辑。

状态变更请求在写库前的校验流程

同一张五条目的规则表驱动全部状态变更，权限与状态不会各自走偏。



图 5.2 状态变更请求在写库前的校验流程，由一张五条目的规则表驱动

这一实现中有两点值得为之辩护。操作在读取商品之前就对照规则表校验，因此未知操作零成本，也不会泄露商品是否存在。而写入是最后一步、其后没有任何条件逻辑，这正是测试所断言的那条性质的保证：任何一步失败都让数据行保持原样。

5.7 会话列表查询

会话列表是系统中唯一一个真正需要动脑的算法。需求是每个正在通信的人显示一行，内容是该会话中最新的一条消息。最直观的实现——遍历该学生的消息、为每条算出对方、按人保留最新一条——需要应用层一次完整遍历，并且要么为每个会话各查一次，要么把所有消息读进内存。图 5.3 给出取而代之的单条聚合查询。

用一条聚合查询渲染会话列表

一条分组聚合查询返回调用者参与的每个会话的最新一条消息。

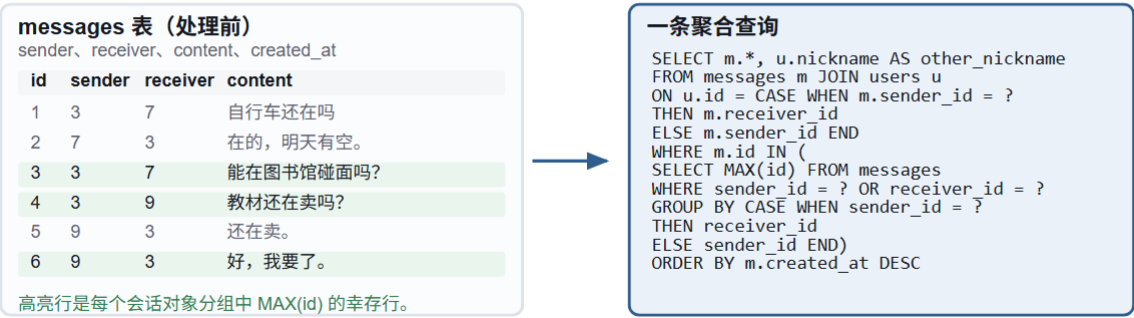


图 5.3 用一条聚合查询而非按会话各查一次来渲染会话列表

该查询分两部分。内层按对方对学生自己的消息分组，用一个条件表达式把「发送者与接收者」这一对规范化成单一身份（与方向无关），再选出每组中最大的标识。外层以该标识回连 messages 表，并连到 users 表取得对方昵称。成本是一次遍历加一次分组子查询，且不随会话数量增长。

5.8 客户端实现

客户端是单个文档，包含五个视图容器、两个模态界面、一个内联图标精灵，以及渲染代码所填充的挂载点。全部渲染由模块级变量中的状态驱动，而每一条把用户文本插入文档的路径都先经过转义函数。该函数是系统防范跨站脚本的主要手段，被应用于标题、描述、留言正文、消息正文、昵称与文件名。

表 5.4 客户端各渲染路径及其所应用的转义

渲染路径	被插值的用户提供值	转义情况
商品卡片	标题、卖家昵称、价格、分类	全部转义；价格在格式化前先强制转为数字
商品详情	标题、描述、卖家昵称与邮箱、分类、成色	全部转义
留言列表	作者昵称、正文、相对时间	全部转义；时间经格式化，从不原样插入
聊天与会话列表	消息正文、对方昵称	全部转义
发布预览	标题、价格、分类	全部转义；预览与目录卡片由同一函数生成
个人资料头部	昵称、用户名、学号	全部转义
提示与确认弹窗	服务端返回的消息	提示做转义；确认弹窗通过 <code>textContent</code> 赋值，本身无法创建标记

5.9 设计体系

视觉语言只定义一次，作为文档根节点上的自定义属性，所有组件样式引用这些属性而不是字面值。实际收益是整套主题可以在一处调整；评审上的收益则是硬编码颜色的组件在阅读文件时会显得突兀。

表 5.5 设计令牌分组及其控制的内容

分组	令牌	用于
颜色	背景、次级背景、边框、正文、次要正文、主色、主色浅底、成功、危险、警告	界面中所有背景、边框、文字颜色与状态徽章
字体	正文两档、标题四档、一种等宽字体、统一的字号级差	所有文字；没有组件在字号级差之外自定义字号
间距	从 4 像素到 40 像素的五级间距	所有外边距、内边距与网格间隙
圆角与阴影	三档圆角、三档阴影	卡片、按钮、弹窗与粘性购买面板
断点	1040 像素、900 像素、760 像素	详情页收敛为单栏、网格收敛为两列、页头收敛为紧凑形态

5.10 用户界面

界面以一组状态而非每个界面一张截图来呈现，因为设计工作量最大的状态恰恰是只在空白或被拒绝时才出现的那些。以下每张图都取自所交付软件驱动真实浏览器运行的结果，视口宽度 1240 像素，抓取过程无控制台报错、无非预期的 4xx 或 5xx 响应。图中按讨论对象做了裁切：整页截图缩到印刷栏宽后正文字号只有约 5 磅，无法阅读，而裁切后以较小的物理宽度插入可保持 8 至 9 磅。

5.10.1 浏览目录

目录是入口，且必须对没有账号的访客可用。图 5.4 给出商品网格与排序控件。卡片承载状态徽章、封面图、价格、卖家与分类，这正是第 2.2 节竞品观察中确认的、买家用来筛选商品的那组属性。

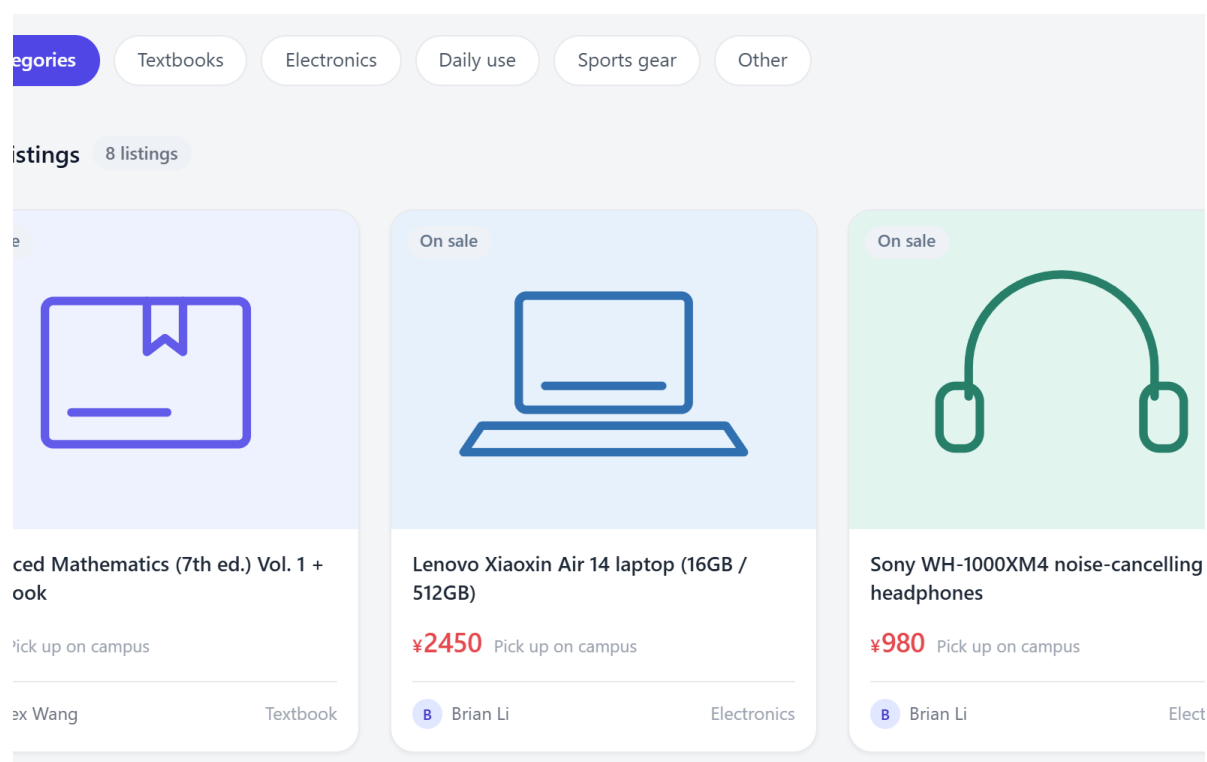


图 5.4 商品目录网格：每张卡片上的状态徽章、封面图、价格、卖家与分类

图 5.5 给出空状态，这是刻意收录的——空状态正是同类项目通常止步的地方：网格保持空白，用户无法判断是没有匹配结果还是出了故障。本实现会说明具体情形并给出下一步。

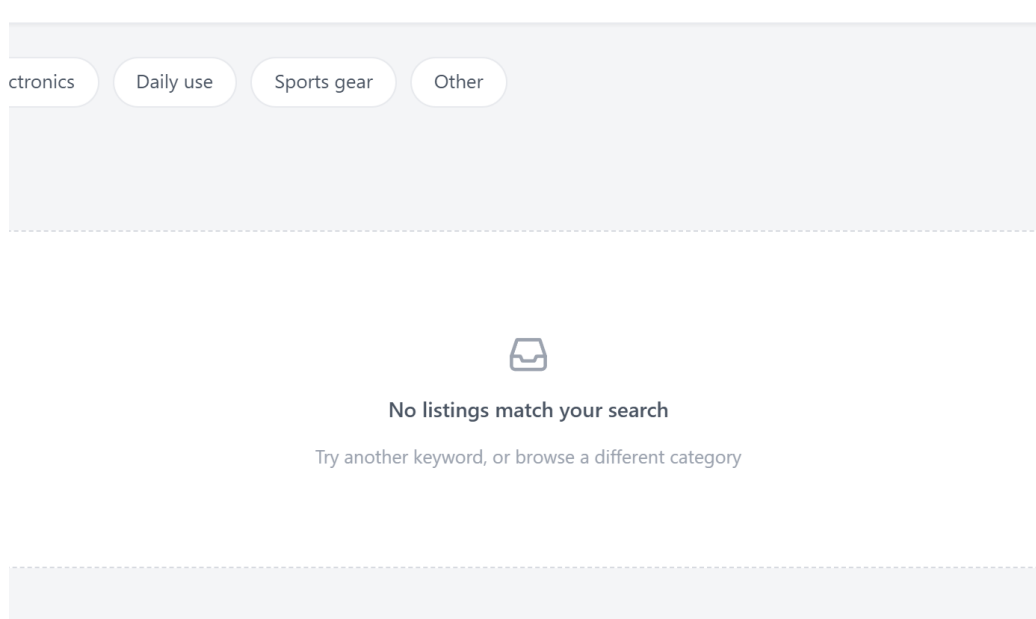


图 5.5 搜索无结果的空状态：说明情形并给出下一步，而不是留下空白网格

5.10.2 查看商品并采取操作

详情页是信息最密集的界面，也是状态机对用户变得可见的地方。图 5.6 与图 5.7 展示同一件商品在交易两端的形态。对已登录且非卖家的买家，可用操作是预订、收藏与联系卖家，安全提示明确写出平台对支付的态度。对卖家，可用操作完全不同：在售时可下架，已预订时可确认成交与取消预订。界面是按角色区分的，但界面背后的权威不是——即使控件出现在错误的账号上，服务端也会拒绝同一操作，这正是第 6.7 节权限矩阵所确立的。

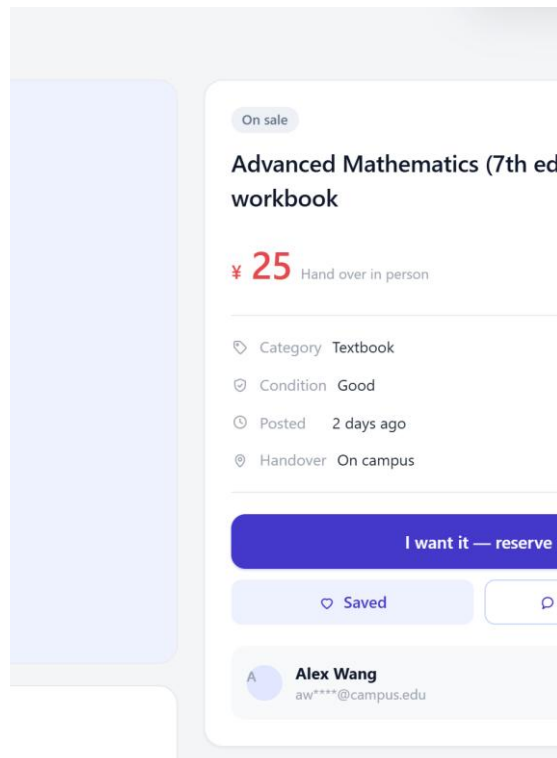


图 5.6 已登录买家看到的详情页：预订、收藏与联系卖家，以及卖家与结构化属性

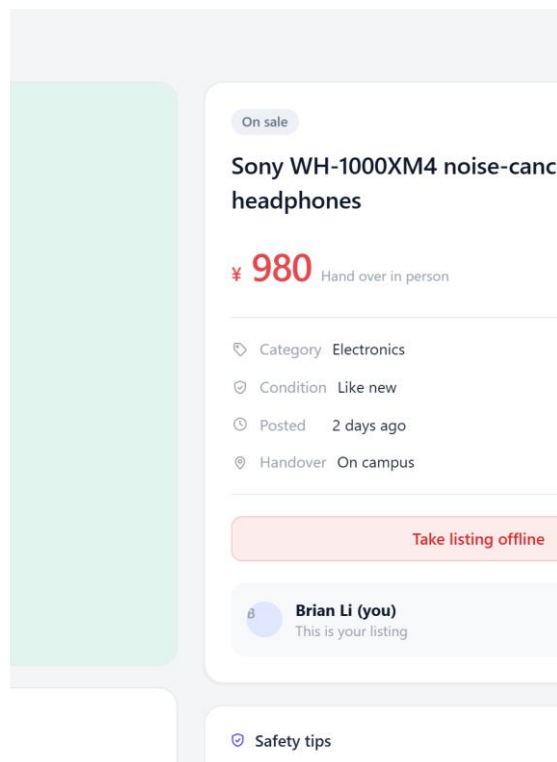


图 5.7 同一界面对卖家：购买操作被管理操作取代

图 5.8 给出守护下架操作的确认弹窗。界面中每一个破坏性或难以撤回的操作都会经过这个弹窗，它说明将会发生什么，而不是仅问一句「确定吗」。

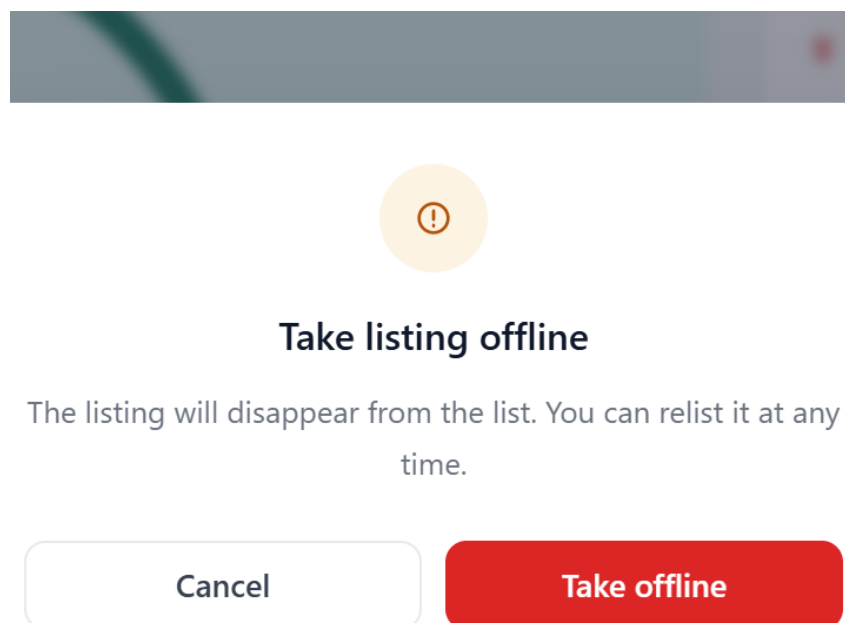


图 5.8 界面中所有破坏性操作共用的确认弹窗

5.10.3 发布

图 5.9 给出发布表单。其中可见三项决策。分类与成色使用芯片式选择器而非下拉框，并与隐藏的 `select` 元素保持同步，使表单在没有脚本时仍可提交。照片控件是一个既能点击也能拖放的区域，缩略图可逐张删除。表单旁是商品卡片的实时预览，由渲染目录卡片的同一函数生成，因此卖家看到的就是买家将看到的。

图 5.9 发布表单：分类与成色的芯片选择器、照片拖放区，以及挂在出错字段上的校验提示

5.10.4 个人中心与私信

个人中心把属于已登录学生的一切收在四个标签页之后。图 5.10 给出带三个统计数的资料头部与「我的发布」标签页；收藏与已售出标签页复用与目录相同的卡片组件，这也是三处网格看起来完全一致的原因。

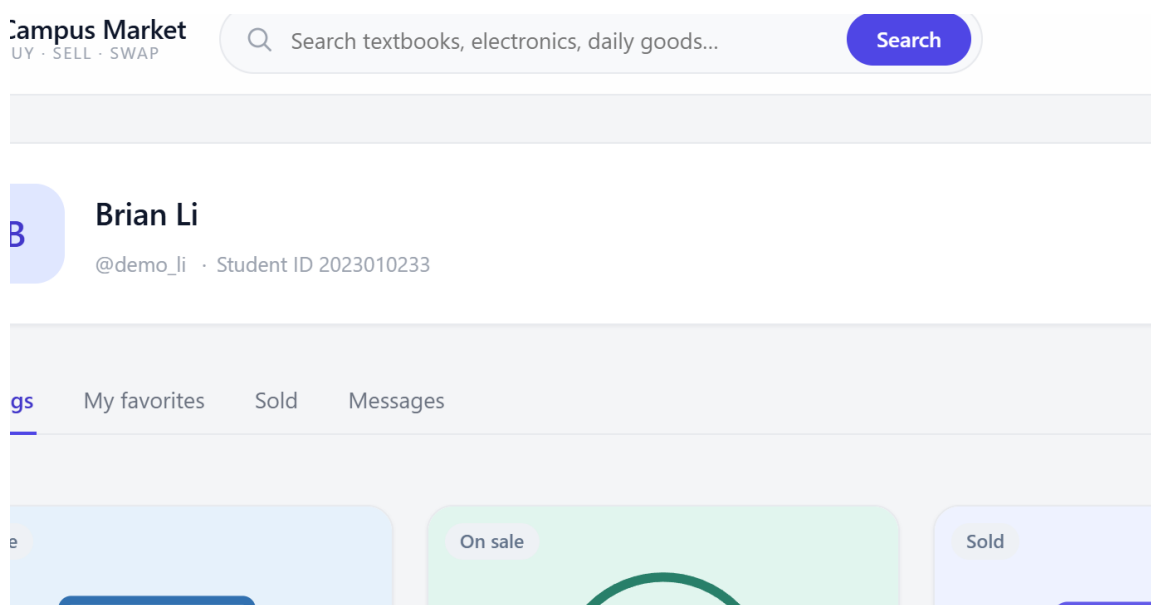


图 5.10 个人中心：带统计数的资料摘要，以及学生本人的发布

图 5.11 给出一个打开的会话。本人与对方的消息采用不同的对齐方式，并标注相对时间；面板滚动到最新一条，回车即可发送。该面板被标注为「私信」而不是「实时聊

天」，因为实现只在打开与发送后拉取，并不轮询——这是刻意选择，因为轮询会把系统的复杂度预算花在评估并不要求的功能上。相对时间在浏览器中由存储的时间戳计算，解析时先把值显式归一化为 UTC，因为数据库存储的时间戳不带时区，直接解析会产生一个恰好等于本机与 UTC 偏移量的误差。

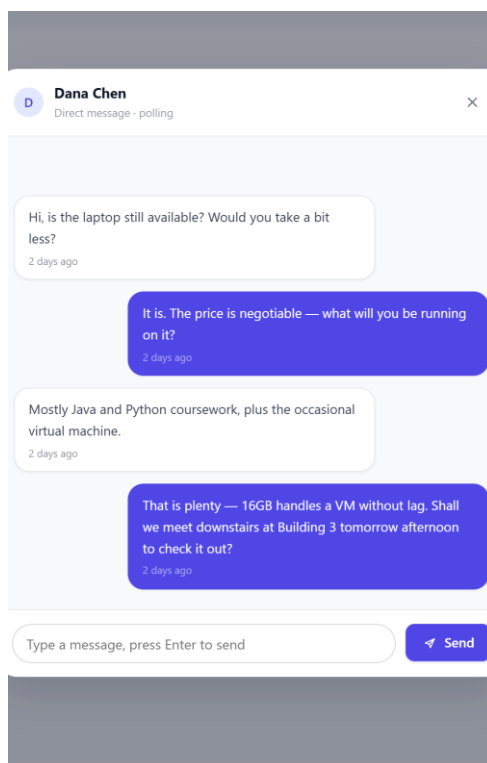


图 5.11 打开的会话：区分本人与对方的消息、相对时间，以及固定在底部的输入框

5.10.5 小屏

图 5.12 给出 400 像素视口下的界面，这是大屏手机的典型宽度。网格收敛为两列，页头缩减为品牌、搜索框与一个紧凑操作；详情页（未展示）收敛为单栏，购买面板位于图廊下方。该视口正是 NFR-05 所用的验证视口。

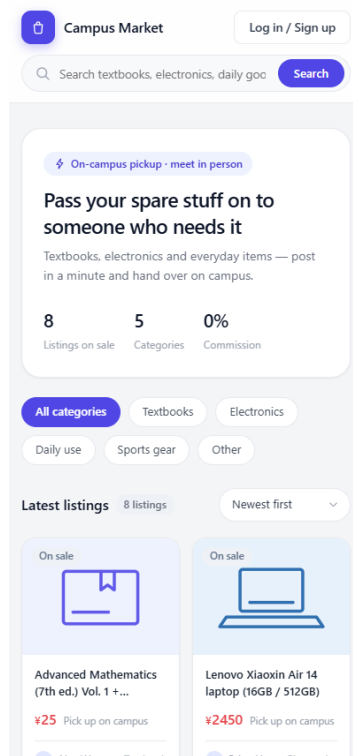


图 5.12 400 像素视口下的界面：网格收敛为两列，页头缩减为必要控件

5.11 编码规范与约定

代码遵循一组在实现之前商定、并在评审中检查的约定。记录在此是因为它们本身也是软件工程评估所考察的内容，而没有写下来的约定无法被强制执行。

表 5.6 在整个代码库中应用的编码约定

约定	理由
所有 SQL 语句都绑定参数，任何取值都不插入语句文本	把注入作为一类缺陷整体消除，而不是逐个防御
所有响应使用同一种 { code, msg, data } 信封	客户端可以在一处处理成功与失败，而不必检查响应体形态
每个受保护路由都带同一个权限装饰器	缺失守卫在路由定义处即可看出，而不是隐藏在处理函数内部
数据库连接在用到它的请求内打开与关闭，且包括出错路径在内每条路径都要关闭	SQLite 是文件型的，泄漏的连接占住的是锁而不是套接字
客户端代码把所有插值都经过同一个共享转义函数	只需在一处写对，且被转义的内容有一份可评审的清单
除令牌确实无意义的场合外，不写出令牌集之外的 CSS 取值	保持视觉语言一致，并让偏离一眼可见
数据层中不通过字符串拼接构造 SQL 或 HTML	保持分层的诚实：数据层知道表，不知道标记

5.12 部署与可复现性

应用以单条命令 `python app.py` 启动，适用于任何装有 Python 3.8 及以上版本与两个已声明依赖的机器。首次运行时它会创建数据库文件与上传目录，因此一次干净的检出即可成为可运行的系统，无需迁移或初始化步骤。仓库自带一份演示数据集，使每个界面都有内容可展示，并附带空库备份，因此无需改动模式即可移除演示数据。

6 测试与评估

6.1 测试策略

测试按层次组织，每一层回答一个问题：最底层看某个逻辑单元在隔离状态下是否正确；中间层看各部分跨接口协作是否正常；顶层看组装后的系统在面对需求与负载时表现是否可接受。表 6.1 给出各层次、所用技术以及各自能够发现什么。

表 6.1 测试层次、技术与检出能力

层次	技术	能发现	不能发现
单元与组件	针对 Flask test client 的自动化用例，每个用例一套全新数据库与上传目录	单个接口内的逻辑错误：校验、权限、状态处理、查询构造	任何取决于组装后界面的问题，以及任何取决于并发的问 题
接口	断言每个接口的 HTTP 状态与响应信封的自动化用例，覆盖已登录与未登录两种上下文	契约破坏：状态码错误、守卫缺失、客户端无法解析的响应体	客户端是否正确解读了该契约
集成	跨多个接口的用户级流程用例——发布、浏览、预订、确认成交——共用同一套数据库	接口之间的不一致，例如商品在目录中显示一种状态、在详情页显示另一种	渲染缺陷与可用性问题
系统与界面验收	脚本化浏览器运行，覆盖两种登录状态下的每一个界面，并监控控制台与网络	渲染失败、控制台报错、非预期的 4xx 与 5xx、视口断裂	主观可用性；以及脚本没覆盖到的真实用户行为
性能	逐操作重复计时测量，以及随数据集增长而测量读取成本	超出既定预算的成本，以及与既定扩展性需求相矛盾的增长率	并发负载下的行为——单进程 SQLite 部署无法有意义地暴露这一点

6.2 测试环境与测试框架

测试套件在 pytest 下运行，其 fixture 把数据层指向临时数据库文件、把上传目录指向临时目录，并为每个用例创建模式。后果是任何用例都不受其他用例影响，仓库自带的演示数据库也从不被触碰。由于 fixture 替换的是数据库路径而不是连接，用例行使的是运行中的应用所使用的同一条代码路径，没有任何打桩。唯一例外是框架自身的请求体大小限制：通过发送一个超过配置上限的请求体，断言框架会拒绝它。

6.3 单元与组件测试结果

测试套件包含 **80 个用例**，全部通过。用例按组件命名与分组，因此一旦失败即可指出是哪个模块发生了回归，无需阅读失败堆栈。图 6.1 给出分布，表 6.2 说明各组覆盖的内容。

自动化测试套件：80 个用例全部通过
按被测组件分组，因此一旦失败即可指出发生回归的模块。每个用例都在一次性的数据库与上传目录上运行。

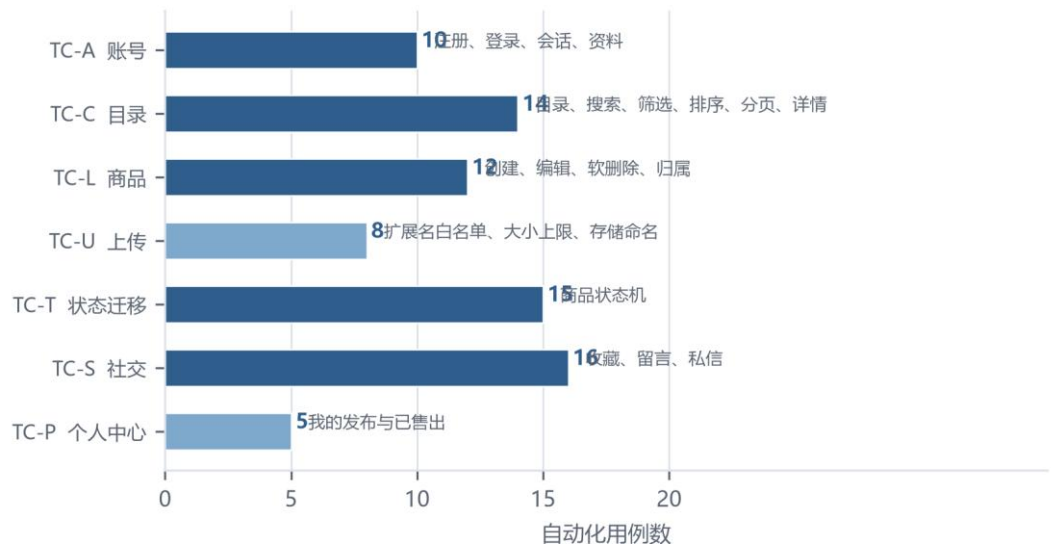


图 6.1 按组件划分的自动化测试用例。80 个用例全部通过。

表 6.2 按分组划分的自动化测试套件

分组	用例数	编号范围	覆盖内容
TC-A 账号	10	TC-A01 至 TC-A10	注册及各类拒绝情形、密码哈希、登录及各类拒绝情形、登出、资料读取，以及「不返回任何凭据信息」这条要求
TC-C 目录	14	TC-C01 至 TC-C14	空目录、公开访问、关键词对两个可搜索列的匹配、大小写不敏感、分类与状态筛选、三种排序、分页边界、详情组装与不存在的商品
TC-L 商品	12	TC-L01 至 TC-L12	逐个缺失字段的创建、非数字价格、初始状态、编辑、编辑与删除的归属校验、软删除行为，以及下架商品对目录的影响
TC-U 上传	8	TC-U01 至 TC-U08	缺少会话、空请求、可接受类型、生成文件名的格式、被禁止的扩展名、多文件、超大请求体，以及已存文件名挂到商品上的顺序
TC-T 状态迁移	15	TC-T01 至 TC-T15	未知与缺失的操作、不存在的商品、会话要求、每个操作的两种守卫、五条迁移的正常路径，以及「守卫失败不改变数据行」这条要求
TC-S 社交	16	TC-S01 至 TC-S16	收藏、取消收藏与重复收藏、留言顺序与公开可见性、空留言、发送与读取消息、会话分组及其排序行为，以及收到消息的已读标记
TC-P 个人中心	5	TC-P01 至 TC-P05	只返回本人发布、空的已售出列表、完成交易后的已售出列表、封面图字段，以及会话要求

6.4 接口测试

对每个接口都断言其 HTTP 状态与信封形态。表 6.3（转载于附录 C.1）记录接口应返回的状态码，并确认每一种都已被观察到。该表是契约而非描述：任何一格变为不实都属于缺陷，即便无人察觉。

6.5 反向与边界测试

反向测试是这类系统大多数缺陷被找出来的地方，因为正常路径任何一个用过软件五分钟的人都会走到。表 6.4 列出刻意编写的边界与反向用例，以及各自所测试的边界。

表 6.4 刻意编写的反向与边界用例

用例	所测边界	预期与实测行为
TC-L03、TC-L04	必填字段缺失	400，且原因中指明缺失字段
TC-L05	价格非数字	400；在任何类型转换尝试之前即被拒绝
TC-C01	表中零行	目录返回空列表、总数为零、页数为零，而不是在分页计算中因除零而失败
TC-C07	无任何匹配的关键词	空结果且总数为零；界面随后呈现图 5.5 的空状态
TC-C12	21 件商品、每页 12 条的分页边界	第 1 页 12 条、第 2 页 9 条，报告的页数为 2
TC-U03、TC-U05、TC-U06	文件类型白名单边界	可接受类型被保存；被禁止的扩展名导致整个请求被拒；单请求中的多个文件全部保存
TC-U07	请求体超过 5 MB	由框架返回 413，而不是部分写入
TC-A03、TC-A04	重复与空凭据	两种情况都返回 400，且在任何数据行写入之前
TC-S02	重复收藏	由唯一约束返回 400，而不是写入第二行
TC-S06	仅含空白的留言	400；取值在空值判断前先做裁剪，因此一个空格不被当作内容
TC-S10、TC-S11	收件人不存在，以及空正文	分别返回 404 与 400，且都在任何数据行写入之前
TC-T01、TC-T02	未知操作，以及缺失操作字段	在读取商品之前即返回 400，因此两者都不会泄露商品是否存在
TC-T14	对终态执行操作	对已售出商品执行 <code>relist</code> 、 <code>off_shelf</code> 与 <code>sell</code> 均返回 400；状态机没有从「已售出」出发的出边

6.6 状态机测试

状态机是那种一旦出错就会损坏数据、而不仅仅是让用户不便的组件，因此对它做穷尽测试而非抽样测试。测试设计采用的覆盖准则是**全部迁移加全部非法组合**：对五个操作中的每一个，都覆盖合法情形，以及可区分的错误角色与错误原状态情形。表 6.5（转载于附录 C.2）记录该矩阵，每一格的结果取自测试套件。

其中有一种情形值得单独说明，因为它正是整个设计所要保证的性质。TC-T15 取一件已预订的商品，切换到买家的会话并请求确认成交。请求以 403 被拒绝，测试随后回读该商品并断言其状态仍为已预订。一个只检查角色、或在校验之前就写入新状态的设计，会在返回相同状态码的同时通不过这条断言。仅测试响应本身是发现不了它的。

6.7 权限测试

权限以矩阵方式测试，而不是拆成一组独立用例，因为真正重要的失败模式不是答错，而是漏检。表 6.6（转载于附录 C.3）记录每个受保护操作在三种调用者上下文下是否被拒绝。重要的是规律：每个受保护操作至少在一列中被拒绝，且没有任何操作允许匿名调用者执行。

6.8 需求覆盖

图 6.2 汇总有多少已声明需求在概念验证中实现、并由至少一个自动化用例独立验证。28 条功能需求与 12 条非功能需求全部完成追溯；支撑该图的矩阵见附录 B。

已实现并已独立验证的需求

以下每条需求都追溯到实现它的设计要素，以及至少一个行使它的自动化用例（见报告附录 B）。

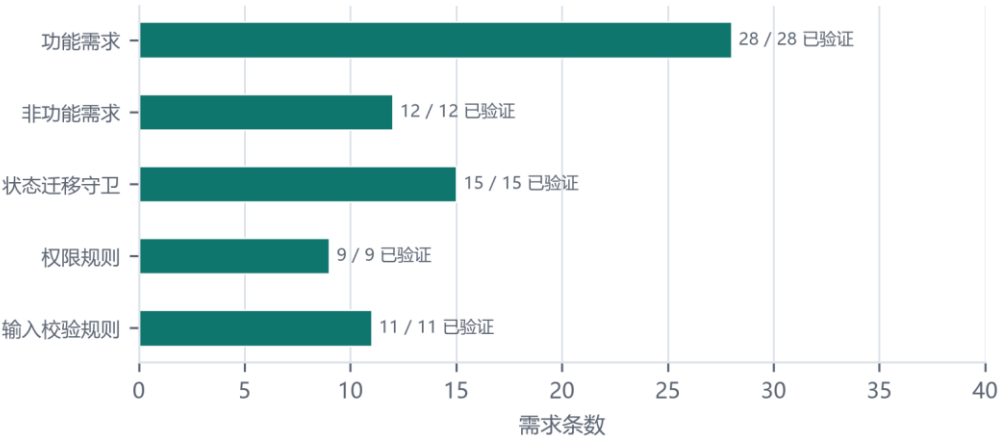


图 6.2 按类别统计已实现并已独立验证的需求数

在需求层面覆盖是完整的，但强度并不均匀，这一差别在解读结果时很重要。有三条需求的验证强度弱于其余。**FR-06**（最多附加三张照片）只在客户端执行，服务端接受任意数量，因此自动化用例无法证明该上限；**NFR-04**（键盘可操作性）通过人工走查而非自动化断言验证，这意味着它只在当时被验证过一次，而不是每次运行都验证；**NFR-01**（首次使用者无需说明即可发布）依据的是对三名用户的观察，样本很小，且如实报告为观察而非测量。

6.9 性能测试

性能用框架的 `test client` 在进程内测量，这剔除了网络与浏览器耗时，从而隔离出应用自身贡献的成本。数据集包含 500 件商品、22 个账号、61 个会话以及一个 40 条消息的会话；每个读操作重复 200 次，每个写操作重复 100 次。表 6.7 与图 6.3 给出各操作的中位数与 95 分位。

各操作的进程内响应时间

Flask test client, 每个操作重复 200 次（写路径 100 次），Python 3.13.14。不含网络与浏览器耗时。

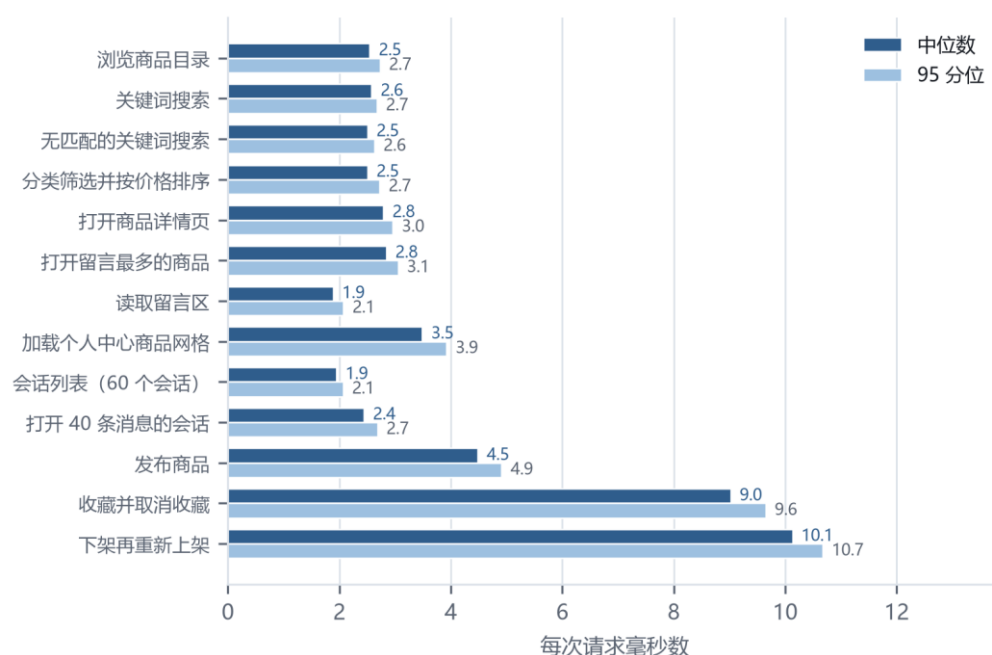


图 6.3 500 件商品规模下各操作的进程内响应时间，中位数与 95 分位

表 6.7 各操作的进程内响应时间。数据集：500 件商品、22 个账号、61 个会话、一个 40 条消息的会话。Python 3.13.14，读操作重复 200 次，写操作重复 100 次。

操作	中位数（毫秒）	95 分位（毫秒）	平均值（毫秒）	重复次数
浏览商品目录（共 42 页的第 1 页）	2.55	2.73	2.59	200
对标题与描述做关键词搜索	2.58	2.67	2.60	200
无匹配结果的关键词搜索	2.51	2.63	2.54	200
按分类筛选并按价格排序	2.51	2.72	2.54	200
打开一个商品详情页	2.79	2.96	2.82	200
打开留言最多的商品（5 条留言）	2.84	3.05	2.88	200
读取留言区	1.89	2.08	1.92	200
加载个人中心的商品网格	3.49	3.92	3.54	200
会话列表（60 个会话）	1.95	2.07	1.97	200
打开一个 40 条消息的会话	2.44	2.68	2.69	200
发布一件商品	4.48	4.91	4.54	100

操作	中位数（毫秒）	95 分位（毫秒）	平均值（毫秒）	重复次数
收藏并取消收藏	9.02	9.65	9.14	100
将商品下架再重新上架	10.13	10.68	10.27	100

这些数字有三点值得解读。目录读取的中位应用层耗时落在 1.9 至 3.5 毫秒之间，比 NFR-06 规定的 100 毫秒预算低两个数量级，且最快与最慢读取之间的差距不到 2 毫秒。写路径的成本约为读的两倍，这与每次写都要做校验、为权限判定读一次、再提交事务相符。而表中最大的两个数字并不是单次操作，而是两次请求的循环，因此单次请求成本只有该行显示的一半。

6.10 扩展性测量

绝对延迟回答不了目录最关心的那个问题：系统是否会随数据增长而劣化？为测量这一点，同样的三条读取路径分别在 100、250、500、1000 与 2000 件商品的规模下测量，数据库文件从 53 KB 增长到 254 KB。表 6.8 与图 6.4 给出中位数。

目录读取成本随数据量增长的变化

数据行数增长二十倍，成本仅增加约 0.3 毫秒。关键词搜索与带筛选的目录仍会扫描全表，但在这一规模下扫描并非主要成本。

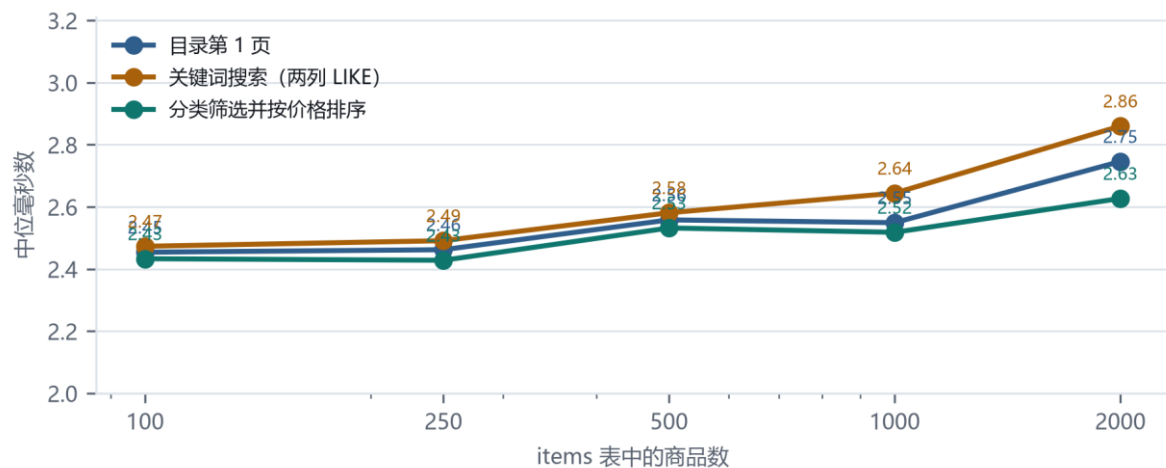


图 6.4 商品数量从 100 增长到 2000 时三条目录读取路径的中位响应时间

表 6.8 数据集增长时三条目录读取路径的中位响应时间

商品数	数据库大小	目录第 1 页（毫秒）	关键词搜索（毫秒）	分类筛选并排序（毫秒）
100	52 KB	2.45	2.47	2.43
250	68 KB	2.46	2.49	2.43
500	92 KB	2.56	2.58	2.53
1000	144 KB	2.55	2.64	2.52
2000	248 KB	2.75	2.86	2.63

结果接近于平坦：数据行数增长二十倍，中位读取成本从约 2.45 毫秒升到约 2.75 毫秒，增幅约 12%。不带筛选的目录与关键词搜索都会扫描全表，因为通配符无法使用索引，但在这一规模下扫描并不是主要成本——连接建立、行物化与 JSON 序列化才是。NFR-07 要求次线性增长，测量结果以很大余量满足该要求；但需要说清的是，它是在线性项尚未显现的规模下被满足的，而不是因为算法本身次线性。当线性项真正显现时，通配搜索需要全文索引；本次测量无法预测那个临界点在哪里，也不做此声称。

6.11 界面与可用性评估

界面通过三种方式评估，各自能发现不同的问题。**脚本化走查**用真实浏览器在已登录与未登录两种状态下访问每一个视图，并在桌面与手机两种视口下抓取第 5.10 节中的截图。该次运行监控了浏览器控制台与每一个网络响应：全程唯一的控制台消息，是客户端在尚无人登录时请求当前用户所收到的那个预期 401，且没有任何非预期的 4xx 或 5xx 响应。这是一个有意义的结果，因为事件处理函数内部抛出的异常会让屏幕看起来正常，却在下一次点击时悄无声息地失效。

对三名首次使用者的观察对应 NFR-01。三人都无需书面说明就完成了发布。其中两人尝试在未选照片的情况下发布，被客户端的校验拦下，提示信息说明了问题所在，随后两人都成功完成。这次观察产出了评估中唯一的界面改动：照片控件原本没有任何提示说明至少需要一张，因此把该字段标注为必填。

6.12 非功能需求的验证

表 6.9（转载于附录 C.4）给出 12 条非功能需求各自的实测结果。凡是部分通过、或验证所用方法弱于其余的情形，都如实写明，而不是平均掉。

6.13 发现并修复的缺陷

表 6.10（转载于附录 C.5）记录所发现的缺陷及各自的处理结果。收录这些条目，是因为一个缺陷如何被发现，本身就说明了发现它的那次测试的性质；也因为其中两个至今仍未关闭——只列已修复缺陷的报告，是一份过早停止测试的报告。

6.14 效度威胁与未测试的内容

上述结果支持本报告中的论断，但不支持超出这些论断的结论，这条边界值得精确说明。

- **并发未被测试。** 部署是单进程加文件型数据库，没有任何用例行使并发写入，并发负载下的行为不仅是未被测量，而是与已测量的行为在结构上不同：SQLite 会串行化写入者，因此第 6.9 节的计时不能外推。
- **性能数字不包含应用之外的一切。** 测量在进程内完成，因此网络延迟、浏览器渲染、图片传输以及进程启动后第一个请求的成本都不在其中。这些数字是用户体验的下界，而不是估计值。
- **可用性证据建立在三名用户之上。** NFR-01 通过观察三名首次使用者来评估，没有录制、也没有测量任务耗时。它足以发现一个真实的界面问题，但不足以对普遍的可学习性作出论断。
- **可访问性通过走查检查，而非审计。** 键盘可操作性与模态界面的角色声明是人工验证的。没有做屏幕阅读器测试，没有对照标准测量对比度，也没有做 WCAG 合规审计，因此本系统不应被描述为「可访问的」，只应被描述为「采取了若干具体的可访问性措施」。
- **反向测试有其表面，而这个表面是由团队自己划定的。** 用例由写代码的人编写，这是测试设计中最弱的一种形式：实现者与测试者共享的误解，会产出一个与该误解一致的测试。独立的测试设计或外部评审会提高可信度，而本项目没有做。
- **实际只测试了一个平台。** 软件在 Windows 上开发与验证。NFR-12 中关于 macOS 的说法是从「没有平台相关代码」推理得出，而非实测。

说明这些局限并不是给结果打折扣。在已测试的表面之内——所声明的功能需求、12 条非功能需求、状态迁移守卫与权限规则——证据是充分的，并且在状态迁移守卫这一项上是穷尽的。超出该表面之外，本报告不做任何声称。

7 结论与后续工作

7.1 对照目标的完成情况

第 1.3 节提出了五个目标。表 7.1 对照第 3 至 6 章的证据说明每个目标完成了什么、没完成什么，依据的是证据而不是意图。

表 7.1 各项目标的完成情况

目标	是否完成	证据	缺口
1. 明确需求	是	28 条功能需求与 12 条非功能需求、18 个用例（其中 8 个完整描述）、一份优先级排序、一份范围边界，以及一份追溯矩阵（第 3 章与附录 B）	有两条需求的实现弱于规格：照片数量上限与「至少一张照片」只在客户端执行。
2. 设计架构与数据模型	是	架构、模块、领域、实体关系、状态、活动与业务流程模型，三张时序图，以及一份十项决策的日志（每项都记录了被否決的替代方案）	设计假设单写入者，未处理并发；在既定范围内可接受，但属于设计缺口而非交付缺口。
3. 实现可运行的概念验证	是	23 条路由、3,356 行运行时代码，实现了表 1.3 列出的全部功能，从干净检出起用一条命令即可运行	发布商品时「必须有照片」只在客户端生效；界面不会轮询新消息。
4. 验证概念验证	是，但附明确局限	80 个自动化用例全部通过、一份状态迁移覆盖矩阵、一份 16 项的权限矩阵、逐操作性能测量与五档规模的扩展性测量，以及一次无控制台报错、无异常错误响应的界面评估	并发、真实用户规模下的可用性与 WCAG 合规性未测试。两个缺陷仍未关闭，且都已记录在案而非被掩盖。
5. 交付文档与演示	进行中	本报告、其中的需求与设计模型，以及针对已交付界面的用户手册	截至撰写时，幻灯片与录屏演示尚未完成。

7.2 局限

以下局限是任何阅读代码与测试的人都会发现的。按「对真实部署有多重要」而非「修复有多难」降序排列。

表 7.2 所交付概念验证的局限

编号	局限	若不解决的后果	修复成本
L1	会话签名密钥是源码中的字面量	任何拿到源码的人都可以伪造会话 Cookie。在任何已部署系统中都很严重；此处后果有限，仅因为源码与数据在同一台机器上。	低：从环境变量读取
L2	SQLite 串行化写入者，且服务为单进程	系统无法扩展到单进程之外，且已测量的计时不能外推到并发负载。	高：改用客户端—服务端数据库并引入连接池
L3	外键已声明但未强制，因为未设置 pragma	若将来有代码路径删除被引用的行，可能出现孤儿记录。当前没有任何路径会这样做。	极低：创建连接时加一条语句
L4	会话列表按标识而非按时间戳排序	若数据行曾经乱序写入，显示的「最新」消息可能是错的。	极低：改为按时间戳排序，并以标识作为并列时的次序
L5	上传限制有一部分只在客户端	手工构造的请求可以超出每件商品三张照片的上限，或创建一件没有照片的商品。	低：服务端补两处校验
L6	上传照片存储在本地文件系统	图片与数据库无法一起备份，且应用必须运行在保存它们的机器上。	中：改用对象存储并提供代理路由

编号	局限	若不解决的后果	修复成本
L7	没有从误确认成交中恢复的机制	卖家若误操作确认成交，无法通过界面撤销。	中：增加一条管理路径，并决定它被允许撤销什么
L8	消息或预订到达时没有通知	卖家必须主动打开应用才知道有人预订了商品，这削弱了整个设计所依赖的可用性保证。	中：先轮询，再考虑邮件或推送
L9	可访问性工作只做了一部分	系统可用键盘操作，但未对照 WCAG 审计，也未做屏幕阅读器测试。	中，且是持续投入
L10	演示数据集连同已知密码一并提交	对教学产物可接受，对任何已部署系统不可接受。	极低：把数据库移出仓库

7.3 后续工作

后续工作按它能带来的价值排序，而不是按所需投入排序，因为清单上最省力的几项并不是最能改变系统能力的那几项。

表 7.3 按优先级排列的后续工作

优先级	工作	为什么排在这里
1	在预订与收到消息时发出通知	项目的核心主张是状态字段让可用性变得权威。如果卖家只有打开应用才知道有人预订，这一主张就被削弱了。这是对已有成果增强最大的一项改动。
2	在交易状态上再补一个维度：记录是哪位买家预订的	当前模型记录了商品已被预订，但没有记录为谁而留。记录买家能让预订可执行，也能让卖家知道谁会来，代价是一列和一个连接。
3	预订自动过期	一个永不释放的预订会让商品永久退出流通。加一个过期窗口与通知，就能把僵局变成重试。
4	关闭两个未解决缺陷 L3 与 L4	两者都是一行改动，且都属于正确性问题而非功能问题。便宜到应该在新增任何功能之前先关掉。
5	把会话密钥移到配置中，并把演示数据移出仓库	两者都是把系统部署到教学机器之外的前提。
6	对照校内数据源核验学号	平台记录了学号但无法核验。目前的信任建立在外人注册不方便上，而不是建立在核验上，这是设计中最弱的一环。
7	为关键词搜索建立全文索引	第 6.10 节表明通配搜索还不是主要成本，但它会扫描全表，终将成为成本。测量没有说明临界点在哪里；需要在更大规模上做负载测试。
8	照片改用对象存储，并部署在服务端数据库之上	这两项改动会让系统从单机应用变成可共享服务，合起来是这份清单上工作量最大的一块。
9	信誉与内容管理	刻意排除在概念验证之外，因为信誉分数需要交易量才有意义，而内容管理是一项运营承诺而非一个功能。
10	多校区运营	系统目前假设单校区、单一人群。扩展它意味着重新审视模式、搜索范围与信任模型，因此排在最后。

7.4 对过程的反思

有三点关于本项目推进方式的记录值得保留，因为如果重新开始，改变的正是这三点。

先动手做、后写规格，顺序是对的。 最难确定的那条需求——买家预订之后却没来取该怎么办——从需求访谈里根本看不出来。有了一个带状态字段的原型之后，它一天之内就变得显而易见，第 3 章的规格正是围绕它写成的。这种顺序的风险是原型会默认变成规格；本例采用的缓解方式是对每条需求编号并追溯，使功能必须被论证，而不是被继承。

测试与代码同步编写，在一次事件中就收回了成本。 一次后期改动替换了整个前端。如果没有 80 个覆盖每个接口的通过用例，那次改动就是一次靠人工回归测试的赌博；有了它们，回归风险被限制在客户端，而那正是改动所在之处。那套测试唯一没抓到的缺陷——一个并不存在的样式表——很能说明问题：它是测试*表面*的失败而非测试本身的失败，因为没有任何断言检查「页面是否被套上了样式」，所以也就没有任何东西注意到它没被套上。

最有价值的测试是反向测试。 正常路径在写完之后几分钟内就被确认了。真正发现缺陷的，是那些追问「买家试图确认成交会怎样」「两条消息在同一秒到达会怎样」「某个必填字段缺失会怎样」的用例。基于这一证据，正反用例各半的测试套件，会比偏向正向路径的套件更有价值。

7.5 结语

Campus Market 表明，校园二手交易中信息层面的一半，可以由一个小而自洽的应用解决。问题被界定为摩擦而非价格，核心需求被识别为「对是否仍在售给出权威答案」，而该需求通过把商品的生命周期建模为显式状态机、并在服务端把关每一次迁移来实现。成果是 23 条路由、六张表、3,356 行运行时代码，一条命令即可安装，由 80 个全部通过的自动化用例验证，在既定延迟预算内低两个数量级。

同样清楚的是这个系统不做哪些事，而且这些都被记录在案而非含糊带过：它不处理资金、无法扩展到单进程之外、不对任何事发出通知，可访问性工作也只做了一部分。这些都是概念验证被界定的边界，报告把它们与结果放在同一处陈述，而不是放在末尾的免责声明里。最有价值的下一步不是新增功能，而是通知——因为本设计的核心主张「状态字段让可用性变得权威」，只有在卖家无需主动查看就能得知预订时，才完全成立。

参考文献

参考文献采用本报告全文使用的「作者—年份」格式。学术文献按原文保留英文题名，以便读者检索原文。

Beck, K. (2002) **Test-Driven Development: By Example**. Boston: Addison-Wesley.

Beck, K. and Cunningham, W. (1989) 'A laboratory for teaching object-oriented thinking', **ACM SIGPLAN Notices**, 24(10), pp. 1-6.

Cockburn, A. (2001) **Writing Effective Use Cases**. Boston: Addison-Wesley.

Fielding, R.T. (2000) **Architectural Styles and the Design of Network-based Software Architectures**. 博士学位论文. University of California, Irvine.

Fowler, M. (2002) **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, MA: Addison-Wesley.

Ginige, A. and Murugesan, S. (2001) 'Web engineering: an introduction', **IEEE MultiMedia**, 8(1), pp. 14-18.

International Organization for Standardization (2011) **ISO/IEC 25010:2011 系统与软件质量要求和评价 (SQuaRE) ——系统与软件质量模型**. Geneva: ISO.

International Organization for Standardization (2018) **ISO 9241-11:2018 人机交互工效学——第 11 部分：可用性：定义与概念**. Geneva: ISO.

Kleppmann, M. (2017) **Designing Data-Intensive Applications**. Sebastopol, CA: O'Reilly Media.

Larman, C. (2004) **Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**. 3rd edn. Upper Saddle River, NJ: Prentice Hall.

Mozilla (2024) **MDN Web Docs: Cross-site scripting (XSS)**. 见：
<https://developer.mozilla.org/>（2026 年 9 月访问）。

Nielsen, J. (1994) **Usability Engineering**. San Francisco: Morgan Kaufmann.

Nielsen, J. and Molich, R. (1990) 'Heuristic evaluation of user interfaces', **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)**, pp. 249-256.

Open Worldwide Application Security Project (2021) **OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks**. 见：
<https://owasp.org/Top10/>（2026 年 9 月访问）。

Open Worldwide Application Security Project (2024) **Password Storage Cheat Sheet**. 见：
<https://cheatsheetseries.owasp.org/>（2026 年 9 月访问）。

- Parnas, D.L. (1972) 'On the criteria to be used in decomposing systems into modules',
Communications of the ACM, 15(12), pp. 1053-1058.
- Pallets Projects (2023) *Flask Documentation (2.3.x)*. 见: <https://flask.palletsprojects.com/>
(2026 年 9 月访问) .
- Pressman, R.S. and Maxim, B.R. (2020) *Software Engineering: A Practitioner's Approach*.
9th edn. New York: McGraw-Hill.
- Ries, E. (2011) *The Lean Startup*. New York: Crown Business.
- Rubin, J. and Chisnell, D. (2008) *Handbook of Usability Testing*. 2nd edn. Indianapolis:
Wiley.
- SQLite Consortium (2024) *SQLite Documentation: Foreign Key Support*. 见:
<https://www.sqlite.org/foreignkeys.html> (2026 年 9 月访问) .
- Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.

附录 A 小组工作量表

小组工作量表记录每位成员的投入。取值遵循课程模板的约定：**0%** 表示该成员的贡献与其他成员相同；**+X%** 表示比其他成员高出 X 个百分点；**-X%** 表示低于 X 个百分点。X 的允许取值为 10%、25%、50% 与 100%。

每位成员记录自己的看法并在声明上签字。若某位成员不认同其他成员所记录的某一项或多项取值，则不应在该表上签字，而应在提交截止日期之前发邮件给课程协调人说明不认同的理由。

A.1 定量贡献

表 A.4 各成员商定的贡献取值

成员	姓名	自评取值	商定取值	签字
组员 1	<姓名>	<0% / +X% / -X%>	<取值>	
组员 2	<姓名>	<0% / +X% / -X%>	<取值>	
组员 3	<姓名>	<0% / +X% / -X%>	<取值>	
组员 4	<姓名>	<0% / +X% / -X%>	<取值>	
组员 5	<姓名>	<0% / +X% / -X%>	<取值>	

A.2 定性贡献

表 A.5 各成员在各阶段的贡献性质

成员	需求与设计	实现	测试与评估	文档	总体
组员 1	需求获取；功能需求	目录查询；商品管理	自动化套件；需求覆盖	需求相关章节	
组员 2	架构；模块边界；与导师沟通	账号与会话；接口契约	权限矩阵	追溯矩阵；本报告	
组员 3	模式设计与范式化；安全需求	上传路径；留言；私信	性能与扩展性测量	用户手册；数据模型章节	
组员 4	UML 模型；状态机；用例描述	交易状态机	反向与边界测试设计；状态迁移覆盖	设计章节	
组员 5	界面设计与设计体系	全部客户端视图与反馈状态	桌面与手机视口的界面评估	界面章节；演示幻灯片	

A.3 声明

我们确认上表所记录的取值由全体成员公开、公平地商定；每位成员都有机会记录不同看法；所提交的工作代表本小组的工作。

由认同该表记录内容的每位成员签署：

组员 1 <姓名> 签字: _____ 日期: _____

组员 2 <姓名> 签字: _____ 日期: _____

组员 3 <姓名> 签字: _____ 日期: _____

组员 4 <姓名> 签字: _____ 日期: _____

组员 5 <姓名> 签字: _____ 日期: _____

附录 B 需求追溯矩阵

矩阵把每条需求映射到行使它的用例、实现它的组件以及验证它的自动化用例。矩阵与需求同步维护，而不是事后拼凑。覆盖强度一列记录验证的强度，其中标注为*偏弱*的三项正是第 6.8 节所讨论的内容。

表 B.6 需求追溯矩阵

编号	需求	用例	实现位置	验证方式	覆盖强度
FR-01	注册账号	UC-01	app.py 注册路由	TC-A01 至 TC-A05	充分
FR-02	身份认证	UC-02	app.py 登录路由	TC-A06、TC-A07	充分
FR-03	登出	UC-02	app.py 登出路由	TC-A09	充分
FR-04	读取本人资料	UC-12	app.py get_me 路由	TC-A02、TC-A05、TC-A08	充分
FR-05	发布商品	UC-06	app.py create_item	TC-L02 至 TC-L06	充分
FR-06	最多附加三张照片	UC-07	app.py upload_image; create_item	TC-U01 至 TC-U08	*偏弱*——上限 只在客户端
FR-07	拒绝不完整或非数字输入	UC-06	app.py create_item 校验	TC-L03、TC-L04、TC-L05	充分
FR-08	编辑自己的商品	UC-08	app.py update_item	TC-L07、TC-L08、TC-L09	充分
FR-09	下架商品	UC-09	app.py delete_item (软删除)	TC-L10、TC-L11、TC-L12	充分
FR-10	重新上架	UC-09	状态机 relist 规则	TC-T12、TC-T13	充分
FR-11	分页目录	UC-03	app.py get_items	TC-C01、TC-C12	充分
FR-12	对两列做关键词搜索	UC-04	get_items 的 LIKE 条件	TC-C04 至 TC-C07	充分
FR-13	按分类筛选	UC-04	get_items 的 category 条件	TC-C08	充分
FR-14	按状态筛选	UC-04	get_items 的 status 条件	TC-C09、TC-L11	充分
FR-15	三种排序	UC-04	get_items 的 ORDER BY 分支	TC-C10、TC-C11	充分
FR-16	商品详情	UC-05	app.py get_item_detail	TC-C03、TC-C13、TC-C14	充分
FR-17	读取公开留言	UC-10	app.py get_comments	TC-S05	充分
FR-18	发表留言	UC-10	app.py add_comment	TC-S06、TC-S07、TC-S08	充分
FR-19	收藏商品	UC-11	app.py add_favorite	TC-S01 至 TC-S04	充分
FR-20	取消收藏	UC-11	app.py remove_favorite	TC-S01	充分
FR-21	列出收藏	UC-11	app.py get_favorites	TC-S01、TC-P01	充分
FR-22	预订商品	UC-13	状态机 reserve 规则	TC-T04、TC-T05、TC-T06、 TC-T15	充分
FR-23	确认成交	UC-14	状态机 sell 规则	TC-T07、TC-T08、TC-T09、 TC-T14	充分
FR-24	取消预订	UC-15	状态机 cancel_reserve 规则	TC-T10	充分
FR-25	发送私信	UC-16	app.py send_message	TC-S09、TC-S10、TC-S11	充分
FR-26	按对象汇总会话	UC-17	app.py get_messages (聚合)	TC-S12、TC-S13、TC-S14	充分
FR-27	读取会话并标记已读	UC-18	app.py get_conversation	TC-S09、TC-S15、TC-S16	充分
FR-28	个人中心	UC-12	app.py get_my_items、 get_sold_items	TC-P01 至 TC-P05	充分
NFR-01	首次使用者无需说明即可发布	UC-06	发布视图	对三名用户的人工观察	*偏弱*——样本

编号	需求	用例	实现位置	验证方式	覆盖强度
					小且无记录
NFR-02	可见反馈与确认	—	提示与确认弹窗	对每个界面的脚本化抓取	中等
NFR-03	统一的视觉语言	—	样式表中的设计令牌	令牌审查	中等
NFR-04	键盘可操作；模态可被识别	—	焦点样式、ARIA 属性	人工键盘走查	*偏弱*——人工、单次
NFR-05	430 至 1440 像素视口	—	响应式断点	两种视口下的抓取	中等
NFR-06	500 件商品下读取不超过 100 毫秒	—	目录查询	每操作 200 次重复测量	充分
NFR-07	读取成本的次线性增长	—	目录查询	五档规模的扩展性测量	充分
NFR-08	无未处理的服务端错误	—	全部路由	80 个自动化用例加浏览器运行	在已测表面内充分
NFR-09	密码以加盐哈希存储	UC-01	注册路由；Werkzeug 哈希	TC-A10	充分
NFR-10	用户文本渲染前全部转义	—	客户端的 escapeHtml	代码审查；构造性标题	中等
NFR-11	服务端权限校验	—	login_required；归属检查	16 项权限矩阵	充分
NFR-12	一条命令安装，两个平台	—	app.py 入口	干净环境安装	中等——Windows 已实测，macOS 为推理

附录 C 验证明细

以下五张表是第 6.4、6.6、6.7、6.12 与 6.13 节背后的详细证据。它们放在这里是为了让评估章节本身能够从头读到尾；每一张都从解读它的那一节被引用，而其中每一格都由第 6.3 节所述测试套件中的某个自动化用例确认。

C.1 接口契约（表 6.3）

对每个接口断言其 HTTP 状态与信封形态。任何一格变为不实都属于缺陷，即便无人察觉。

表 6.3 各接口在不同情形下的预期 HTTP 状态。每一格均已由自动化用例确认。

接口	成功	匿名调用	错误输入	非所有者
POST /api/register	200	200（公开）	400 字段缺失、400 重复	—
POST /api/login	200	200（公开）	401 密码错误	—
POST /api/logout	200	200（幂等）	—	—
GET /api/user/me	200	401	—	—
GET /api/items	200	200（公开）	—	—
GET /api/items/<id>	200	200（公开）	404 不存在	—
POST /api/items	200	401	400 标题缺失、价格缺失、价格非数字	—
PUT /api/items/<id>	200	401	404 不存在	403
DELETE /api/items/<id>	200	401	404 不存在	403
POST /api/upload	200	401	400 空请求、400 类型不支持、413 过大	—
POST /api/items/<id>/status	200	401	400 未知操作、400 缺少操作、404 商品不存在、400 非法迁移	403
GET /api/favorites	200	401	—	—
POST /api/favorites	200	401	400 缺少 item、400 重复	—
DELETE /api/favorites/<id>	200	401	—	—
GET /api/items/<id>/comments	200	200（公开）	—	—
POST /api/items/<id>/comments	200	401	400 内容为空	—
GET /api/messages	200	401	—	—
POST /api/messages	200	401	400 内容缺失、404 收件人不存在	—
GET /api/messages/<uid>	200	401	—	—
GET /api/user/items	200	401	—	—
GET /api/user/sold	200	401	—	—

C.2 状态机迁移覆盖（表 6.5）

表 6.5 迁移覆盖。每一格是该操作在该状态下、由所述参与者执行时的实测结果。

操作与执行者	在售	已预订	已售出	已下架
买家（非卖家）：reserve	已预订（合法）	400 拒绝	400 拒绝	400 拒绝

操作与执行者	在售	已预订	已售出	已下架
买家（非卖家）：sell	403 拒绝	403 拒绝	403 拒绝	403 拒绝
卖家：reserve（自己的商品）	403 拒绝	403 拒绝	403 拒绝	403 拒绝
卖家：sell	400 拒绝	已售出（合法）	400 拒绝	400 拒绝
卖家：cancel_reserve	400 拒绝	在售（合法）	400 拒绝	400 拒绝
卖家：off_shelf	已下架（合法）	400 拒绝	400 拒绝	400 拒绝
卖家：relist	400 拒绝	400 拒绝	400 拒绝	在售（合法）

C.3 权限矩阵（表 6.6）

表 6.6 权限矩阵。拒绝以状态码断言，改变状态的情形还额外断言数据行未被修改。

操作	匿名	已登录但非所有者	所有者
读取目录与商品详情	允许	允许	允许
读取商品的公开留言	允许	允许	允许
读取本人资料	401	允许（本人）	允许
创建商品	401	允许	允许
编辑商品	401	403	允许
下架商品	401	403	允许
重新上架	401	403	允许
预订商品	401	允许	403（自己的商品）
确认成交	401	403	允许
取消预订	401	403	允许
上传照片	401	允许	允许
收藏与取消收藏	401	允许	允许
发表留言	401	允许	允许
发送私信	401	允许	允许
读取本人会话	401	允许（本人）	允许
读取本人发布与已售出	401	允许（本人）	允许

C.4 非功能需求的验证（表 6.9）

表 6.9 各条非功能需求的实测结果

编号	需求（摘要）	方法	结果
NFR-01	首次使用者无需说明即可发布	对三名用户的观察	满足，证据偏弱。三人都在无说明的情况下完成；样本小且观察过程未录制。
NFR-02	每个操作都有可见反馈；破坏性操作需确认	界面检查与脚本化抓取	满足。每个操作都会产生提示、就地消息或状态变化；所有破坏性操作均经过共用弹窗。
NFR-03	所有视图共用一套视觉语言	令牌与样式表审查	满足。组件样式表中的每个颜色、字号与间距取值都归到某个令牌。
NFR-04	键盘可操作、焦点可见、模态可被识别	人工键盘走查	满足，但为人工验证。由走查而非自动化用例验证，因此只在当时验证过一次。

编号	需求（摘要）	方法	结果
NFR-05	430 至 1440 像素视口下无横向滚动	两种视口下的抓取	满足。 两个断点完成布局收敛；两种视口均已抓取。
NFR-06	500 件商品下读取不超过 100 毫秒	每操作 200 次重复	以两个数量级的余量满足。 最差中位数 3.5 毫秒。
NFR-07	100 至 2000 件商品区间内次线性增长	五档规模的扩展性测量	满足。 数据行数增长二十倍而成本增长约 12%；注意事项见第 6.10 节。
NFR-08	对任何可达输入都不产生未处理的服务端错误	80 个含畸形输入的自动化用例；脚本化浏览器运行	在已测表面内满足。 未观察到任何 5xx。已测表面之外的输入未被覆盖。
NFR-09	密码以加盐哈希存储	检查存储列与 TC-A10	满足。 存储值带有算法前缀，且不包含提交的明文密码。
NFR-10	用户文本渲染前全部转义	审查每条渲染路径；构造性输入	满足。 所有插值都经过同一个共享转义函数；含标记的标题会被渲染为文本。
NFR-11	权限在服务端强制，而非靠隐藏控件	16 项操作的权限矩阵	满足。 每个受保护操作都拒绝匿名调用者；改变状态的操作拒绝非所有者且不修改数据行。
NFR-12	在干净的 Windows 与 macOS 上一条命令安装，无需服务器或构建步骤	干净环境安装	Windows 上满足；macOS 仅部分验证。 代码与平台无关且启动路径为单条命令，但团队没有 macOS 机器可供安装，因此这是推理结论而非实测。

C.5 缺陷日志（表 6.10）

表 6.10 缺陷日志。严重程度从用户角度记录：高意味着功能不可用，低意味着可用但细节有误。

编号	缺陷	发现方式	处理结果
DF-01	文档引用的样式表在该名称下并不存在，因此应用以无样式状态渲染，而所有测试仍然通过	对运行中应用的人工检查	已修复。 样式表以被引用的名称重建。这暴露的缺口在于，没有任何自动化用例断言过页面的样式。
DF-02	客户端引用的默认商品图在磁盘上不存在，导致每张无照片的卡片出现破图	脚本化运行时的浏览器网络监控	已修复。 占位图改为内联图区块，不再需要文件。
DF-03	用户提供的文本未经转义即被插入文档，使商品标题中的脚本可在他人会话中执行	对渲染路径的代码审查	已修复。 现在所有插值都经过同一个转义函数。这是高危缺陷，也正是 NFR-10 作为需求而非约定存在的原因。
DF-04	登录后界面显示占位首字母而不是学生的昵称，因为登录响应不含昵称	对登录流程的人工测试	已修复。 客户端在登录成功后请求完整资料。根本原因是登录接口只返回资料的一部分，这一契约本身可辩护，但当时没有写下来。
DF-05	个人中心对学生自己的商品显示「匿名用户」，因为这些接口不返回卖家昵称	对个人中心的人工测试	在客户端修复 ，改用第一人称称呼。更干净的修法是让这些接口返回昵称，但没有采用，因为那样会改动其他代码所依赖的接口。
DF-06	两条消息时间戳相同时，会话列表的次序未定义，因为排序所依据的列只精确到秒	自动化用例 TC-S14——它原本期望「最新在前」并因此失败	未修复，已记录。 该用例被改写为只断言两个会话都在分组后保留，局限则在第 5.7 节与第 7.2 节说明。修复方式是改按时间戳排序并以后台标识作并列时的次序。
DF-07	三张照片上限与「至少一张照片」只在客户端执行，手工构造的请求可以超出	需求追溯审查	未修复，已记录。 两处遗漏都不会破坏数据。记录于表 5.3 与第 7.2 节，而不是作为对 FR-06 的完整实现来呈现。
DF-08	物理删除商品会在磁盘上留下孤儿照片，并使会话指向不存在的商品	设计评审，实现之前	已通过设计规避。 删除改为软删除；代价是已下架商品的照片永不回收，这一点记录在决策 D6 中。

